HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BỘ MÔN HỆ THỐNG THÔNG MINH



ASSIGMENT-3

Giảng viên hướng dẫn	**: PGS.TS.Trần Đình Quế**
Họ và tên sinh viên	**: Trần Quang Lâm**
Mã sinh viên	**: B23DCCN480**
Lớp	**: D23CQCN04-B**

Hà Nội – 2026

CHƯƠNG I. BẢN CHẤT LÝ THUYẾT VÀ TỰ CÀI ĐẶT MẠNG NƠ-RON 1
1. Bản chất toán học của Học sâu: Phép hợp các hàm tham số hóa 2
1.1. So sánh mô hình truyền thống và mô hình sâu 2
1.2. Cấu trúc toán học bên trong một nơ-ron và một tầng ẩn..... 2
2. Phân biệt Đặc trưng tự thiết kế (Hand-crafted Features) và Biểu diễn học được (Learned Representation) 3
3. Chu trình huấn luyện 4 bước 3
4. Đối chiếu cài đặt: NumPy From Scratch so với PyTorch Framework 4
CHƯƠNG II. THIẾT KẾ KIẾN TRÚC MẠNG HỌC SÂU 5 TẦNG VÀ PHÂN TÍCH TENSOR SHAPES..... 6
1. Tổng quan kiến trúc mạng MLP 5 tầng (5-Layer Deep Neural Network Architecture)..... 6
2. Bảng đặc tả chi tiết kích thước Tensor và Không gian tham số 6
3. Cấu hình chi tiết layer 5 cho 3 bài toán thực tế[cite: 2, 3, 4]..... 7
3.1. Bài toán 1 – Chẩn đoán Lâm sàng Đái tháo đường (Diabetes Prediction - Binary Classification):[cite: 2, 4] 7
3.2 Bài toán 2 – Định giá Bất động sản (House Price Prediction - Regression):..... 7
3.3 Bài toán 3 – Khai phá Hành vi & Sở thích Khách hàng Tiki (Tiki Interest - Multiclass Classification):[cite: 2, 3]..... 8
3.4 Giải thích bản chất phân cấp biểu diễn (Hierarchical Representation Learning)..... 8
CHƯƠNG III. TIẾN HÀNH THỰC NGHIỆM ĐỐI SÁNH 9
A. Bài toán 1 Diabetes Prediction 9
1. Thiết lập thực nghiệm và quy trình triển khai từng bước..... 9
B. Bài toán 2 House Price Prediction 16
1. Thiết lập thực nghiệm hồi quy từng bước 16
C. Bài toán 3 Tiki Prediction..... 21
1. Thiết lập thực nghiệm đa lớp từng bước 21
Bảng 3.3. Bảng đối sánh hiệu năng phân loại sở thích Tiki trên Test Set 26

CHƯƠNG IV. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU DIỄN HỌC ĐƯỢC 27
A. BÀI TOÁN 1: DIABETES PREDICTION 28
1. Phân tích biểu đồ đường cong huấn luyện 28
2. Phân tích Ma trận nhầm lẫn (Confusion Matrix) trên Tập kiểm thử 29
3. So sánh khoa học: Học máy truyền thống so với Học sâu 5 tầng 30
B. BÀI TOÁN 2: HOUSE PRICE 30
1. Phân tích biểu đồ đường cong huấn luyện hồi quy 31
2. Phân tích đồ thị tương quan giữa Giá thực tế và Giá dự đoán (Scatter Plot trên thang Log) 32
3. Lý giải nguyên nhân bùng nổ sai số thực tế (MAE, RMSE) và Giải pháp khắc phục 33
C. BÀI TOÁN 3 TIKI PREDICTION 33
1. Phân tích đường cong huấn luyện Cross-Entropy 34
2. Phân tích Ma trận nhầm lẫn Đa lớp (Confusion Matrix) 34
3. Thảo luận khoa học: Vì sao Deep Learning lại chiếm ưu thế vượt trội trên Bài toán Tiki? 35
CHƯƠNG V. TỔNG KẾT VÀ ĐÁNH GIÁ TOÀN DIỆN 36
1. Tổng kết kết quả thực nghiệm qua 3 bài toán nghiên cứu 36
Bảng 5.1. Bảng tổng hợp hiệu năng đối sánh toàn diện giữa Classical ML và Deep Learning 36
2. Đúc kết các quy luật khoa học về Mạng nơ-ron sâu (Core Technical Insights) 37
3. Hạn chế của hệ thống và Hướng phát triển tiếp theo 37

CHƯƠNG I. BẢN CHẤT LÝ THUYẾT VÀ TỰ CÀI ĐẶT MẠNG NƠ-RON

1. Bản chất toán học của Học sâu: Phép hợp các hàm tham số hóa

Một trong những tư tưởng cốt lõi xuyên suốt học phần là: Học sâu không phải là một chiếc hộp đen thần bí, mà bản chất là một chuỗi các phép biến đổi toán học tham số hóa được kết hợp tuần tự với nhau.

1.1. So sánh mô hình truyền thống và mô hình sâu

Trong học máy truyền thống, mối quan hệ giữa dữ liệu đầu vào và kết quả đầu ra thường được mô hình hóa qua một ánh xạ trực tiếp duy nhất:

$$\hat{y} = f(X)$$

Ngược lại, mạng nơ-ron sâu (Deep Learning) không ánh xạ trực tiếp một bước từ dữ liệu thô sang nhãn dự đoán. Thay vào đó, mô hình xây dựng một chuỗi phân cấp gồm nhiều tầng biến đổi trung gian:

$$X \rightarrow f_1(X) \rightarrow f_2(f_1(X)) \rightarrow f_3(f_2(f_1(X))) \rightarrow ... \rightarrow f_k(...)$$

Dưới góc độ toán học giải tích, đầu ra của mạng nơ-ron k tầng là phép hợp hàm (Function Composition):

$$\hat{y} = (f_k \circ ... \circ f_2 \circ f_1)(X)$$

Chú thích các thành phần toán học:

- X: Tensor hoặc ma trận dữ liệu đầu vào có kích thước B × d (trong đó B là kích thước lô - batch size, d là số chiều đặc trưng gốc).
- f_l: Hàm biến đổi phi tuyến tại tầng thứ l (với l = 1, 2, ..., k).
- ∘: Ký hiệu toán học của phép toán hợp hàm (Composition operator)
- ŷ: Giá trị dự đoán đầu ra của mô hình (xác suất phân loại hoặc giá trị liên tục hồi quy).

1.2. Cấu trúc toán học bên trong một nơ-ron và một tầng ẩn

Mỗi tầng biến đổi $\mathbf{f_l}$ được cấu thành từ hai giai đoạn toán học nối tiếp nhau:

- Phép biến đổi tuyến tính affine (Linear Transformation): Chiếu không gian từ chiều d sang không gian mới k chiều thông qua phép nhân ma trận trọng số và cộng vector độ chệch:

  $$Z = XW + b$$

  Trong đó: W ∈ ℝ^(d × k) là ma trận trọng số liên kết; b ∈ ℝ^(1 × k) là vector hệ số chệch bias
- Phép kích hoạt phi tuyến (Nonlinear Activation):

  $H = \phi(Z)$

  Trong đó: $\phi(\cdot)$ là hàm kích hoạt phi tuyến, điển hình là hàm ReLU(z) = max(0, z)

Ý nghĩa nguyên lý: Nếu không có hàm kích hoạt phi tuyến $\phi(\cdot)$, việc xếp chồng bao nhiêu tầng tuyến tính cũng chỉ tương đương với một phép nhân ma trận duy nhất:

$W_2W_1X + (W_2b_1 + b_2) = W_tổng\ X + b_tổng$. Do đó, sự kết hợp giữa Độ sâu (Depth) và Tính phi tuyến (Nonlinearity) là điều kiện bắt buộc giúp mạng nơ-ron học được các không gian biểu diễn phức tạp.

2. Phân biệt Đặc trưng tự thiết kế (Hand-crafted Features) và Biểu diễn học được (Learned Representation)

Deep Learning: Data ——> Learned Representation $h_\theta(x)$ (Mô hình tự tối ưu) ——> Prediction

Tiêu chí so sánh	Đặc trưng tự thiết kế - Traditional ML	Biểu diễn học được- Deep Learning
Bản chất trích xuất	Con người (kỹ sư dữ liệu) tự phân tích miền tri thức, lựa chọn cột, tạo tương tác thủ công	Mạng nơ-ron tự động khám phá và tạo ra biểu diễn thông qua quá trình tối ưu hóa hàm mất mát.
Dạng biểu diễn toán học	Vector đặc trưng ban đầu $x = [x_1, x_2, ..., x_d]$ với các trường rõ ràng (tuổi, huyết áp, diện tích...).	Vector tiềm ẩn tại tầng ẩn: $h = [h_1, h_2, ..., h_k] = ReLU(W_1x + b_1)$.
Ý nghĩa ngữ nghĩa	Có ý nghĩa vật lý trực tiếp, dễ đọc hiểu và giải thích cho con người (High Interpretability).	Là các chiều trừu tượng trong không gian Euclid đa chiều, không có nhãn ý nghĩa định tính trực tiếp.
Cơ chế phân cấp	Phẳng (Flat): Toàn bộ đặc trưng được ép chung vào một vector đầu vào duy nhất.	Phân cấp (Hierarchical): Tầng 1 học biểu diễn sơ cấp từ dữ liệu thô, tầng 2 học "biểu diễn từ biểu diễn".
Khả năng thích ứng	Cố định, không tự thay đổi khi phân phối nhãn mục tiêu thay đổi	Linh hoạt: Các trọng số $\theta = \{W, b\}$ tự điều chỉnh theo gradient để tối ưu riêng cho từng bài toán.

Bảng 1.1. So sánh chi tiết giữa Đặc trưng tự thiết kế và Biểu diễn học được

3. Chu trình huấn luyện 4 bước

Quá trình học tập của mạng nơ-ron là một chu trình khép kín lặp lại liên tục qua từng lô dữ liệu (mini-batch) và từng chu kỳ (epoch):

Input Batch → Forward → Loss → Backward (Chain Rule) → Optimizer Step (Update)

Bước 1: Lan truyền tiến (Forward Propagation)

- Dữ liệu đầu vào X được truyền qua từng tầng để tính toán lần lượt các vector tiềm ẩn và tạo ra vector điểm số đầu ra (logits) Z_output:
  $Z_1 = XW_1 + b_1$; $H_1 = ReLU(Z_1)$
  $Z_2 = H_1W_2 + b_2$; $H_2 = ReLU(Z_2)$
  $Z_3 = H_2W_3 + b_3$; $\hat{y} = \sigma(Z_3)$ (hoặc giữ nguyên logits cho hàm mất mát)

Bước 2: Đo lường sai số bằng hàm mất mát (Loss Calculation)

- Hàm mất mát $\mathscr{L}(\theta)$ đo lường mức độ sai lệch giữa dự đoán của mạng $\hat{y}$ và nhãn mục tiêu thực tế y:
  - Đối với bài toán phân lớp nhị phân (Binary Classification):
    $$\mathscr{L} = -(1/N) \sum [\, y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \,]$$
  - Đối với bài toán phân lớp đa lớp (Multi-class Classification):
    $$\mathscr{L} = -(1/N) \sum \sum y_ic \ln(p_ic)$$

Bước 3: Lan truyền ngược theo quy tắc dây chuyền

- Mục tiêu là tính đạo hàm riêng của hàm mất mát đối với từng ma trận trọng số và bias: $\nabla_W \mathscr{L} = \partial\mathscr{L} / \partial W$.
- Theo quy tắc dây chuyền giải tích (Chain Rule), gradient từ tầng cuối cùng được truyền ngược dần về các tầng trước:
  - Tại tầng ra: Sai số nhạy cảm $dZ_3 = \hat{y} - y$.
  - Gradient tham số tầng ra: $dW_3 = H_2^T \cdot dZ_3$ và $db_3 = \sum dZ_3$
  - Truyền gradient về tầng ẩn trước: $dH_2 = dZ_3 \cdot W_3^T$.
  - Đạo hàm qua hàm kích hoạt phi tuyến: $dZ_2 = dH_2 \odot ReLU'(Z_2)$.
  - Gradient tham số tầng ẩn: $dW_2 = H_1^T \cdot dZ_2$ và $db_2 = \sum dZ_2$.

Bước 4: Cập nhật tham số theo hướng giảm độ dốc

- Sau khi đã xác định được gradient, thuật toán tối ưu hóa (Gradient Descent) sẽ điều chỉnh các tham số theo chiều ngược lại của gradient để cực tiểu hóa hàm mất mát:
  - $W \leftarrow W - \eta \cdot (\partial\mathscr{L} / \partial W)$
  - $b \leftarrow b - \eta \cdot (\partial\mathscr{L} / \partial b)$

  Trong đó: η (learning rate) là tốc độ học quy định độ lớn của mỗi bước nhảy cập nhật tham số).

4. Đối chiếu cài đặt: NumPy From Scratch so với PyTorch Framework

Việc tự cài đặt mạng nơ-ron bằng NumPy thuần giúp hiểu cặn kẽ từng phép nhân ma trận và dòng chảy gradient. Trong khi đó, việc chuẩn hóa bằng PyTorch giúp tự động hóa tính toán đạo hàm và tối ưu tài nguyên phần cứng (GPU/TPU).

Thao tác kỹ thuật	Cài đặt thủ công bằng NumPy (From Scratch)	Chuẩn hóa trong Framework PyTorch

1. Quản lý dữ liệu theo lô	Tự tính chỉ số xáo trộn indices = np.random.permutation() và cắt slice mảng.	Sử dụng đối tượng TensorDataset kết hợp DataLoader(batch_size=32, shuffle=True)
2. Khởi tạo tham số	Tự tạo mảng: W1 = np.random.randn(8, 16) * np.sqrt(2.0/8) (He Normal).	Tự động hóa hoàn toàn khi khai báo các khối nn.Linear(8, 16).
3. Lan truyền tiến	Viết tường minh: z1 = X @ W1 + b1; h1 = np.maximum(0, z1)	Khai báo kiến trúc trong nn.Sequential hoặc phương thức forward(self, x).
4. Lưu vết tính toán	Bắt buộc phải tạo từ điển cache = {"X": X, "z1": z1, ...} để dùng cho backward	Hệ thống Autograd của PyTorch tự động xây dựng đồ thị tính toán động
5. Tính hàm mất mát	Tự cài đặt công thức toán: loss = -np.mean(y*log(y_hat) + (1-y)*log(1-y_hat))	Gọi hàm định sẵn: criterion = nn.BCEWithLogitsLoss() hoặc nn.CrossEntropyLoss().
6. Lan truyền ngược	Tự giải tích đạo hàm bằng tay và code ma trận: dz3 = (y_hat - y)/n; dW3 = H2.T @ dz3	Chỉ cần gọi một lệnh duy nhất: loss.backward().
7. Cập nhật tham số	Phải trừ thủ công từng biến: W1 -= lr * gradients["dW1"].	Xóa gradient cũ và cập nhật: optimizer.zero_grad(); optimizer.step().
8. Chuyển đổi trạng thái	Không hỗ trợ tự động.	Tách biệt rõ ràng giữa chế độ học và suy luận: model.train() và model.eval() với torch.no_grad().

Bảng 1.2. Bảng đối chiếu song song giữa NumPy thuần và PyTorch

Kết luận bản chất: Framework PyTorch không hề làm thay đổi nền tảng toán học của học sâu. PyTorch đóng vai trò là một bộ công cụ tự động hóa đạo hàm (Automatic Differentiation) và tối ưu hóa tính toán tensor song song trên phần cứng, giải phóng kỹ sư khỏi việc tính vi phân thủ công cho các mạng hàng trăm tầng phức tạp.

CHƯƠNG II. THIẾT KẾ KIẾN TRÚC MẠNG HỌC SÂU 5 TẦNG VÀ PHÂN TÍCH TENSOR SHAPES

1. Tổng quan kiến trúc mạng MLP 5 tầng (5-Layer Deep Neural Network Architecture)

Theo yêu cầu thực nghiệm, mô hình học sâu được xây dựng dưới dạng mạng Perceptron đa tầng (Multi-Layer Perceptron - MLP) bao gồm đúng 5 tầng biến đổi có tham số học được (5 trainable layers). Mạng thực hiện biến đổi tuần tự từ không gian vector đặc trưng đầu vào d chiều qua các tầng ẩn nhằm trích xuất các biểu diễn tiềm ẩn phân cấp, trước khi ánh xạ về không gian đầu ra tương ứng cho từng bài toán:

$$X \in \mathbb{R}^{(B \times d)} \xrightarrow{(f_1)} H_1 \in \mathbb{R}^{(B \times 128)} \xrightarrow{(f_2)} H_2 \in \mathbb{R}^{(B \times 64)} \xrightarrow{(f_3)} H_3 \in \mathbb{R}^{(B \times 32)} \xrightarrow{(f_4)} H_4 \in \mathbb{R}^{(B \times 16)} \xrightarrow{(f_5)} \text{Output (ŷ hoặc Logits)}$$

B (Batch Size): Kích thước của lô dữ liệu huấn luyện

d (Input Dimension): Số chiều của vector đặc trưng đầu vào sau khi tiền xử lý

f_1, f_2, f_3, f_4: Các tầng ẩn phi tuyến (Hidden Layers) sử dụng phép biến đổi tuyến tính kết hợp hàm kích hoạt ReLU.

f_5: Tầng đầu ra (Output Layer) ánh xạ không gian tiềm ẩn về giá trị dự đoán nhiệm vụ.

2. Bảng đặc tả chi tiết kích thước Tensor và Không gian tham số

Bảng dưới đây mô tả chi tiết dòng chảy tensor qua từng tầng, kích thước ma trận trọng số, vector hệ số chệch bias và công thức tính số lượng tham số cần tối ưu hóa:

Tầng (Layer)	Phương trình biến đổi toán học	Kích thước Trọng số (W)	Kích thước Bias (b)	Tensor Đầu ra (Output Shape)	Hàm kích hoạt	Số lượng tham số (Params)
Input Layer	Lô dữ liệu đầu vào: X			B × d		0
Layer 1	$Z_1 = X \cdot W_1 + b_1 H_1 = \text{ReLU}(Z_1)$	$W_1 \in \mathbb{R}^{(d \times 128)}$	$b_1 \in \mathbb{R}^{(1 \times 128)}$	B × 128	ReLU	d × 128 + 128
Layer 2	$Z_2 = H_1 \cdot W_2 + b_2 H_2 = \text{ReLU}(Z_2)$	$W_2 \in \mathbb{R}^{(128 \times 64)}$	$b_2 \in \mathbb{R}^{(1 \times 64)}$	B × 64	ReLU	128 × 64 + 64 = 8.256

Layer 3	$Z_3 = H_2 \cdot W_3 + b_3 H_3 = \text{ReLU}(Z_3)$	$W_3 \in \mathbb{R}^{(64 \times 32)}$	$b_3 \in \mathbb{R}^{(1 \times 32)}$	B × 32	ReLU	64 × 32 + 32 = 2.080
Layer 4	$Z_4 = H_3 \cdot W_4 + b_4 H_4 = \text{ReLU}(Z_4)$	$W_4 \in \mathbb{R}^{(32 \times 16)}$	$b_4 \in \mathbb{R}^{(1 \times 16)}$	B × 16	ReLU	32 × 16 + 16 = 528
Layer 5 (Output)	$Z_5 = H_4 \cdot W_5 + b_5 \hat{y} = g(Z_5)$	$W_5 \in \mathbb{R}^{(16 \times \text{Out})}$	$b_5 \in \mathbb{R}^{(1 \times \text{Out})}$	B × Out	Tùy bài toán	16 × Out + Out

3. Cấu hình chi tiết layer 5 cho 3 bài toán thực tế[cite: 2, 3, 4]

Do 3 tập dữ liệu có bản chất nhiệm vụ học máy khác nhau, tầng thứ 5 (Output Layer) được thiết kế chuyên biệt như sau:

3.1. Bài toán 1 – Chẩn đoán Lâm sàng Đái tháo đường (Diabetes Prediction - Binary Classification):[cite: 2, 4]

- Số nơ-ron đầu ra: Out = 1.
- Kích thước tham số: Ma trận trọng số $W_5 \in \mathbb{R}^{(16 \times 1)}$, vector hệ số chệch $b_5 \in \mathbb{R}^{(1 \times 1)}$.
- Tensor đầu ra: $Z_5 \in \mathbb{R}^{(B \times 1)}$ (chứa các giá trị logit không bị chặn biên).
- Kỹ thuật cài đặt PyTorch: Mạng nơ-ron không tích hợp hàm Sigmoid trực tiếp ở cuối mô hình mà trả về logit thuần, sau đó truyền vào hàm mất mát nn.BCEWithLogitsLoss(). Cách tiếp cận này kết hợp tính toán Sigmoid và Binary Cross-Entropy thành một bước duy nhất bằng kỹ thuật số học Log-Sum-Exp, giúp triệt tiêu hoàn toàn lỗi tràn số (overflow/underflow) khi giá trị logit quá lớn hoặc quá nhỏ. Khi suy luận (Inference), xác suất được tính qua công thức: $p = \sigma(Z_5) = 1 / (1 + e^{(-Z_5)})$

3.2 Bài toán 2 – Định giá Bất động sản (House Price Prediction - Regression):

- Số nơ-ron đầu ra: Out = 1.
- Kích thước tham số: Ma trận trọng số $W_5 \in \mathbb{R}^{(16 \times 1)}$, vector hệ số chệch $b_5 \in \mathbb{R}^{(1 \times 1)}$.
- Tensor đầu ra: $\hat{y} = Z_5 \in \mathbb{R}^{(B \times 1)}$ (Giá trị dự đoán liên tục của biến mục tiêu ln(1 + price)).

- Hàm kích hoạt: Tuyến tính đơn thuần (Identity / Linear Activation - không sử dụng hàm phi tuyến ở tầng ra) để mô hình có thể dự báo các mức giá trị số thực trên toàn miền (-∞, +∞).
- Hàm mất mát: Sai số bình phương trung bình nn.MSELoss(): $\mathscr{L} = (1 / B) \sum (y_i - \hat{y}_i)^2$[cite: 2, 3]

3.3 Bài toán 3 – Khai phá Hành vi & Sở thích Khách hàng Tiki (Tiki Interest - Multiclass Classification):[cite: 2, 3]

- Số nơ-ron đầu ra: Out = C = 5 (tương ứng 5 danh mục ngành hàng sở thích)[cite: 2, 3].
- Kích thước tham số: Ma trận trọng số $W_5 \in \mathbb{R}^{(16 \times 5)}$, vector hệ số chệch $b_5 \in \mathbb{R}^{(1 \times 5)}$.
- Tensor đầu ra: $Z_5 \in \mathbb{R}^{(B \times 5)}$ (Vector chứa 5 điểm số logits đại diện cho 5 lớp).
- Kỹ thuật cài đặt PyTorch: Mô hình xuất ra trực tiếp vector Logits và kết nối với hàm mất mát nn.CrossEntropyLoss(). Trong quá trình suy luận, phân phối xác suất được chuẩn hóa qua hàm Softmax và nhãn dự đoán được xác định theo nguyên lý cực đại hóa xác suất: $p_c = e^{(z_c)} / \sum e^{(z_j)}$ và $\hat{y} = \text{argmax}(Z_5)$

3.4 Giải thích bản chất phân cấp biểu diễn (Hierarchical Representation Learning)

Mạng nơ-ron sâu 5 tầng hoạt động như một hệ thống trích xuất đặc trưng tự động có cấu trúc phân cấp tuần tự:

Tầng 1 (Layer 1: $d \rightarrow 128$) – Học các tổ hợp đặc trưng sơ cấp (Primary Feature Combinations):

- Tầng này tiếp nhận trực tiếp vector thuộc tính thô $x \in \mathbb{R}^d$. Mỗi nơ-ron trong số 128 nơ-ron tính toán một tích vô hướng $w^T x + b$, đóng vai trò như một siêu phẳng định hướng trong không gian dữ liệu.
- Thông qua hàm kích hoạt $\text{ReLU}(z) = \max(0, z)$, tầng 1 học cách phát hiện 128 mẫu kết hợp tuyến tính cục bộ cơ bản (ví dụ: mối liên hệ đồng thời giữa mức đường huyết cao và độ tuổi trung niên, hoặc diện tích mặt sàn lớn đi kèm mặt tiền rộng).

Tầng 2, 3 và 4 (Layers 2, 3, 4: $128 \rightarrow 64 \rightarrow 32 \rightarrow 16$) – Học biểu diễn từ biểu diễn (Learning Representations of Representations):

- Đây là hạt nhân bản chất của Học sâu (Deep Learning). Tầng 2 không tiếp nhận dữ liệu gốc X mà tiếp nhận không gian biểu diễn $H_1 \in \mathbb{R}^{(B \times 128)}$ do tầng 1 tạo ra. Tiếp đó, tầng 3 nhận $H_2 \in \mathbb{R}^{(B \times 64)}$, và tầng 4 nhận $H_3 \in \mathbb{R}^{(B \times 32)}$.
- Bằng cách xếp chồng liên tiếp các phép biến đổi phi tuyến:

$$H_2 = \text{ReLU}(H_1 \cdot W_2 + b_2)$$

$$H_3 = \text{ReLU}(H_2 \cdot W_3 + b_3)$$

$$H_4 = \text{ReLU}(H_3 \cdot W_4 + b_4)$$

- Mạng nơ-ron tiến hành tổng hợp các mẫu sơ cấp thành các khái niệm trừu tượng cấp cao hơn (Abstract Concepts). Không gian dữ liệu được "nén" dần về mặt số chiều (từ 128 → 64 → 32 → 16) nhưng tăng dần về mật độ thông tin hữu ích. Quá trình này giúp uốn cong, kéo giãn và phân tách các cụm dữ liệu vốn đan xen phức tạp ở không gian thô ban đầu thành một không gian tiềm ẩn $H_4 \in \mathbb{R}^{(B \times 16)}$ có tính phân tách tuyến tính rất cao (Linearly Separable Latent Space).

Tầng 5 (Output Layer: 16 → Out) – Ánh xạ không gian tiềm ẩn về nhãn dự đoán (Task Mapping):

- Tầng cuối cùng thực chất đóng vai trò như một bộ phân loại tuyến tính (Linear Classifier) hoặc hồi quy tuyến tính đơn giản đặt ngay trên không gian biểu diễn cấp cao H_4.
- Khi vector biểu diễn H_4 đã được tối ưu hóa đầy đủ, tầng 5 chỉ việc thiết lập siêu phẳng quyết định để đưa ra kết quả dự đoán cuối cùng một cách chính xác.

CHƯƠNG III. TIẾN HÀNH THỰC NGHIỆM ĐỐI SÁNH

A. Bài toán 1 Diabetes Prediction

1. Thiết lập thực nghiệm và quy trình triển khai từng bước

Quá trình thực nghiệm được triển khai tuần tự qua 6 bước kỹ thuật tương ứng với cấu trúc các khối lệnh (cells) trên môi trường Jupyter Notebook:

- Bước 1 – Nạp và tiền xử lý dữ liệu chống rò rỉ:

```python
# ========================================================================
# CELL 0: TỰ ĐỘNG NẠP FILE TỪ 'dataset clinical' VÀ CHUẨN BỊ X_train, y_train
# ========================================================================
import os
import glob
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Tìm đường dẫn file csv trong thư mục 'dataset clinical'
clinical_dir = "dataset clinical"
csv_files = glob.glob(os.path.join(clinical_dir, "*.csv"))

if not csv_files:
    raise FileNotFoundError(f"Không tìm thấy file .csv nào trong thư mục '{clinical_dir}'!")

data_path = csv_files[0]
print(f"• Đang nạp dữ liệu từ tệp: {data_path}")
df = pd.read_csv(data_path)

# 2. Xác định biến mục tiêu (target)
target_col = "diabetes" if "diabetes" in df.columns else df.columns[-1]

# 3. Mã hóa biến phân loại (nếu có chuỗi như gender, smoking_history)
df_processed = pd.get_dummies(df, drop_first=True)

# Tách ma trận đặc trưng X và nhãn y
X = df_processed.drop(columns=[target_col]).values
y = df_processed[target_col].values

# 4. Phân chia tập dữ liệu: Train (70%), Val (15%), Test (15%) có Stratified
X_train_raw, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
X_val_raw, X_test_raw, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)
# 5. Chuẩn hóa Z-score (Chỉ fit trên Train để chống rò rỉ dữ liệu)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_val = scaler.transform(X_val_raw)
X_test = scaler.transform(X_test_raw)

# 6. Cập nhật số chiều d thực tế
INPUT_DIM = X_train.shape[1]

print("=== [CELL 0] XỬ LÝ DỮ LIỆU THÀNH CÔNG ===")
print(f"• Kích thước X_train : {X_train.shape} (Số chiều input d = {INPUT_DIM})")
print(f"• Kích thước X_val   : {X_val.shape}")
print(f"• Kích thước X_test  : {X_test.shape}")
```

```
• Đang nạp dữ liệu từ tệp: dataset clinical/diabetes_dataset.csv
=== [CELL 0] XỬ LÝ DỮ LIỆU THÀNH CÔNG ===
• Kích thước X_train : (70000, 73) (Số chiều input d = 73)
• Kích thước X_val   : (15000, 73)
• Kích thước X_test  : (15000, 73)
```

- Đọc tập dữ liệu diabetes_dataset.csv từ thư mục dataset clinical/.
- Áp dụng mã hóa biến phân loại One-Hot Encoding (pd.get_dummies(..., drop_first=True)), mở rộng không gian đặc trưng từ các cột thô ban đầu lên thành d = 73 chiều.
- Phân chia dữ liệu theo chiến lược phân tầng (Stratified Split) để bảo toàn tỷ lệ nhãn bệnh: tập Huấn luyện (Train) gồm 70.000 mẫu (70%), tập Thẩm định (Validation) gồm 15.000 mẫu (15%), và tập Kiểm thử độc lập (Test) gồm 15.000 mẫu (15%).
- Chuẩn hóa thang đo Z-Score bằng StandardScaler. Tuân thủ nghiêm ngặt nguyên tắc chống rò rỉ dữ liệu (Data Leakage Prevention): hàm fit_transform chỉ được tính toán trên 70.000 mẫu của tập Train, sau đó áp dụng phép biến đổi transform sang tập Val và Test.

- Bước 2 – Đóng gói PyTorch Tensor và tạo DataLoader mini-batch:

```python
# ============================================================================
# CELL 1: KHỞI TẠO TENSOR & DATALOADER (MINI-BATCH)
# ============================================================================
import numpy as np
import torch
from torch.utils.data import DataLoader, TensorDataset

# Cố định seed ngẫu nhiên để bảo toàn tính tái lập (Reproducibility)
torch.manual_seed(42)
np.random.seed(42)

# Chuyển đổi dữ liệu từ numpy array sang PyTorch Tensor
train_dataset = TensorDataset(
    torch.tensor(X_train, dtype=torch.float32),
    torch.tensor(y_train, dtype=torch.float32).view(-1, 1),
)
val_dataset = TensorDataset(
    torch.tensor(X_val, dtype=torch.float32),
    torch.tensor(y_val, dtype=torch.float32).view(-1, 1),
)
test_dataset = TensorDataset(
    torch.tensor(X_test, dtype=torch.float32),
    torch.tensor(y_test, dtype=torch.float32).view(-1, 1),
)

# Tạo DataLoader mini-batch (Batch Size = 64)
BATCH_SIZE = 64
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

# In thông tin kiểm tra kích thước
print("=== [CELL 1] THÔNG TIN NẠP DỮ LIỆU TENSOR ===")
print(
    f"• Số mẫu Train : {len(train_dataset)} | Số batches: {len(train_loader)}"
)
print(f"• Số mẫu Val   : {len(val_dataset)}   | Số batches: {len(val_loader)}")
print(f"• Số mẫu Test  : {len(test_dataset)}  | Số batches: {len(test_loader)}")

# Lấy thử 1 batch đầu tiên kiểm tra tensor shape
sample_X, sample_y = next(iter(train_loader))
print(
    f"• Kích thước lô kiểm tra (Batch Shape) -> X: {sample_X.shape}, y: {sample_y.shape}"
)
```

```python
# Lấy thử 1 batch đầu tiên kiểm tra tensor shape
sample_X, sample_y = next(iter(train_loader))
print(
    f"• Kích thước lô kiểm tra (Batch Shape) -> X: {sample_X.shape}, y: {sample_y.shape}"
)
```

```
=== [CELL 1] THÔNG TIN NẠP DỮ LIỆU TENSOR ===
• Số mẫu Train : 70000 | Số batches: 1094
• Số mẫu Val   : 15000   | Số batches: 235
• Số mẫu Test  : 15000  | Số batches: 235
• Kích thước lô kiểm tra (Batch Shape) -> X: torch.Size([64, 73]), y: torch.Size([64, 1])
```

- Chuyển đổi toàn bộ mảng NumPy thành kiểu dữ liệu torch.float32 và đóng gói vào cấu trúc TensorDataset.
- Cố định hạt giống ngẫu nhiên (torch.manual_seed(42), np.random.seed(42)) nhằm bảo đảm tính tái lập kết quả.
- Thiết lập kích thước lô mini-batch cố định B = 64. Tập Train gồm 70.000 mẫu được chia thành 1.094 batches, tập Val và Test mỗi tập được chia thành 235 batches. Kích hoạt cờ shuffle = True cho tập Train để xáo trộn ngẫu nhiên thứ tự mẫu sau mỗi chu kỳ huấn luyện.
- Kích thước tensor kiểm tra đầu vào: X ∈ ℝ^(64 × 73) và nhãn mục tiêu y ∈ ℝ^(64 × 1).

- Bước 3 – Khởi tạo kiến trúc mạng nơ-ron sâu 5 tầng (Deep MLP Architecture):

```python
# ================================================================================
# CELL 2: ĐỊNH NGHĨA KIẾN TRÚC MẠNG MLP 5 TẦNG
# ================================================================================
import torch.nn as nn


class DeepMLP5Layers(nn.Module):

  def __init__(self, input_dim=12, output_dim=1):
    super(DeepMLP5Layers, self).__init__()
    self.network = nn.Sequential(
        nn.Linear(input_dim, 128),  # Layer 1: d -> 128
        nn.ReLU(),
        nn.Linear(128, 64),  # Layer 2: 128 -> 64
        nn.ReLU(),
        nn.Linear(64, 32),  # Layer 3: 64 -> 32
        nn.ReLU(),
        nn.Linear(32, 16),  # Layer 4: 32 -> 16
        nn.ReLU(),
        nn.Linear(16, output_dim),  # Layer 5: 16 -> Out
    )

  def forward(self, x):
    return self.network(x)


# Khởi tạo mô hình kiểm tra
test_model = DeepMLP5Layers(input_dim=12, output_dim=1)

# Tính tổng số lượng tham số học được
total_params = sum(p.numel() for p in test_model.parameters() if p.requires_grad)

print("=== [CELL 2] KIẾN TRÚC VÀ THAM SỐ MẠNG 5 TẦNG ===")
print(test_model)
print("-------------------------------------------------")
print(f"• Tổng số tham số cần huấn luyện: {total_params:,} tham số.")
print("• Dòng chảy Tensor: 12 -> 128 -> 64 -> 32 -> 16 -> 1")
```

```
=== [CELL 2] KIẾN TRÚC VÀ THAM SỐ MẠNG 5 TẦNG ===
DeepMLP5Layers(
  (network): Sequential(
    (0): Linear(in_features=12, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=64, bias=True)
    (3): ReLU()
    (4): Linear(in_features=64, out_features=32, bias=True)
    (5): ReLU()
    (6): Linear(in_features=32, out_features=16, bias=True)
    (7): ReLU()
    (8): Linear(in_features=16, out_features=1, bias=True)
  )
)
-------------------------------------------------
• Tổng số tham số cần huấn luyện: 12,545 tham số.
• Dòng chảy Tensor: 12 -> 128 -> 64 -> 32 -> 16 -> 1
```

- Định nghĩa lớp mô hình kế thừa từ nn.Module gồm 5 tầng biến đổi tuyến tính kết hợp hàm kích hoạt phi tuyến ReLU:
  - Tầng 1 (Layer 1): Tuyến tính 73 → 128, kích hoạt ReLU.
  - Tầng 2 (Layer 2): Tuyến tính 128 → 64, kích hoạt ReLU.
  - Tầng 3 (Layer 3): Tuyến tính 64 → 32, kích hoạt ReLU.
  - Tầng 4 (Layer 4): Tuyến tính 32 → 16, kích hoạt ReLU.
  - Tầng 5 (Layer 5): Tuyến tính 16 → 1 (xuất logits trực tiếp).
- Tổng số lượng tham số học được của toàn bộ mạng được xác định chính xác là 20.353 tham số.

- Bước 4 – Xây dựng hàm huấn luyện và thẩm định chuẩn 4 bước:

```python
# ============================================================================
# CELL 3: ĐỊNH NGHĨA HÀM HUẤN LUYỆN (TỰ ĐỘNG KHỚP INPUT DIM)
# ============================================================================
import torch.optim as optim

def train_and_evaluate(lr=0.001, epochs=40, opt_type="Adam"):
    # Tự động lấy chính xác số chiều đặc trưng của X_train (ở đây là 73)
    input_dim = X_train.shape[1]

    model = DeepMLP5Layers(input_dim=input_dim, output_dim=1)
    criterion = nn.BCEWithLogitsLoss()

    if opt_type == "Adam":
        optimizer = optim.Adam(model.parameters(), lr=lr)
    else:
        optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)

    train_losses, val_losses = [], []
```

```python
    for epoch in range(1, epochs + 1):
        # Training mode
        model.train()
        running_train_loss = 0.0
        for X_b, y_b in train_loader:
            optimizer.zero_grad()
            outputs = model(X_b)
            loss = criterion(outputs, y_b)
            loss.backward()
            optimizer.step()
            running_train_loss += loss.item() * X_b.size(0)

        epoch_train_loss = running_train_loss / len(train_loader.dataset)
        train_losses.append(epoch_train_loss)

        # Validation mode
        model.eval()
        running_val_loss = 0.0
        with torch.no_grad():
            for X_b, y_b in val_loader:
                outputs = model(X_b)
                val_loss = criterion(outputs, y_b)
                running_val_loss += val_loss.item() * X_b.size(0)

        epoch_val_loss = running_val_loss / len(val_loader.dataset)
        val_losses.append(epoch_val_loss)

        if epoch % 10 == 0 or epoch == 1:
            print(f"  Epoch [{epoch:02d}/{epochs}] | Train Loss: {epoch_train_loss:.4f} | Val Loss: {epoch_val_loss:.4f}")

    return model, train_losses, val_losses

print("=== [CELL 3] ĐÃ CẬP NHẬT HÀM TRAIN THEO SỐ CHIỀU INPUT ĐỘNG ===")
print(f"• Số chiều Layer 1 sẽ tự động nhận: {X_train.shape[1]} -> 128")
```

```
=== [CELL 3] ĐÃ CẬP NHẬT HÀM TRAIN THEO SỐ CHIỀU INPUT ĐỘNG ===
• Số chiều Layer 1 sẽ tự động nhận: 73 -> 128
```

- Thiết lập chu trình huấn luyện khép kín cho từng mini-batch theo đúng 4 giai đoạn giải tích:
    - Giai đoạn 1: Lan truyền tiến (Forward) outputs = model(X_b).
    - Giai đoạn 2: Tính tổn thất bằng hàm mất mát kết hợp nn.BCEWithLogitsLoss().
    - Giai đoạn 3: Tính đạo hàm riêng theo quy tắc chuỗi (Backward) loss.backward().
    - Giai đoạn 4: Cập nhật trọng số theo hướng giảm gradient optimizer.step() sau khi đã xóa gradient cũ bằng optimizer.zero_grad().
    - Tách biệt chế độ vận hành: chuyển sang model.train() khi huấn luyện và model.eval() đi kèm khối torch.no_grad() khi tính toán tổn thất thẩm định trên tập Validation.
- Bước 5 – Thực nghiệm kiểm soát tối ưu (Learning Rate Experiment) và Lưu mô hình:

```python
# ============================================================================
# CELL 4: CHẠY 2 THỰC NGHIỆM TỐI ƯU HÓA VÀ LƯU FILE .PTH
# ============================================================================
import os

print("=== [CELL 4] BẮT ĐẦU THỰC NGHIỆM KIỂM SOÁT TỐC ĐỘ HỌC ===")

print("\n--- Thực nghiệm 1: Bộ tối ưu Adam với Learning Rate = 0.001 ---")
model_adam_001, train_l_001, val_l_001 = train_and_evaluate(
    lr=0.001, epochs=40, opt_type="Adam"
)

print("\n--- Thực nghiệm 2: Bộ tối ưu Adam với Learning Rate = 0.01 ---")
model_adam_01, train_l_01, val_l_01 = train_and_evaluate(
    lr=0.01, epochs=40, opt_type="Adam"
)

# Lưu mô hình tốt nhất (.pth)
os.makedirs("models", exist_ok=True)
save_path = "models/mlp_5layers_diabetes.pth"
torch.save(model_adam_001.state_dict(), save_path)

print(f"\n• Đã lưu thành công trọng số mô hình tốt nhất vào tệp: '{save_path}'")
```

```
=== [CELL 4] BẮT ĐẦU THỰC NGHIỆM KIỂM SOÁT TỐC ĐỘ HỌC ===

--- Thực nghiệm 1: Bộ tối ưu Adam với Learning Rate = 0.001 ---
  Epoch [01/40] | Train Loss: 0.1391 | Val Loss: 0.1179
  Epoch [10/40] | Train Loss: 0.0753 | Val Loss: 0.1046
  Epoch [20/40] | Train Loss: 0.0499 | Val Loss: 0.1501
  Epoch [30/40] | Train Loss: 0.0323 | Val Loss: 0.2131
  Epoch [40/40] | Train Loss: 0.0231 | Val Loss: 0.2901

--- Thực nghiệm 2: Bộ tối ưu Adam với Learning Rate = 0.01 ---
  Epoch [01/40] | Train Loss: 0.1394 | Val Loss: 0.1262
  Epoch [10/40] | Train Loss: 0.0929 | Val Loss: 0.1049
  Epoch [20/40] | Train Loss: 0.0803 | Val Loss: 0.1053
  Epoch [30/40] | Train Loss: 0.0673 | Val Loss: 0.1222
  Epoch [40/40] | Train Loss: 0.0566 | Val Loss: 0.1426

• Đã lưu thành công trọng số mô hình tốt nhất vào tệp: 'models/mlp_5layers_diabetes.pth'
```

- Tiến hành huấn luyện song song hai kịch bản tối ưu hóa qua 40 Epochs để đối sánh ảnh hưởng của siêu tham số tốc độ học:
    - Kịch bản 1 (Learning Rate $\eta = 0.001$): Bộ tối ưu Adam đạt độ hội tụ sâu, Train Loss giảm mạnh từ 0.1391 (Epoch 1) xuống 0.0231 (Epoch 40).
    - Kịch bản 2 (Learning Rate $\eta = 0.01$): Tốc độ học lớn khiến quá trình cập nhật bị dao động quanh cực tiểu, Train Loss tại Epoch 40 chỉ đạt mức 0.0566.
    - Lưu trữ trọng số của mô hình tối ưu nhất (Kịch bản $\eta = 0.001$) vào đường dẫn models/mlp_5layers_diabetes.pth bằng lệnh torch.save(model.state_dict(), ...) theo đúng yêu cầu bài tập.

- Bước 6 – Đánh giá kiểm thử và Trực quan hóa kết quả:

```python
# ============================================================================
# CELL 5: ĐÁNH GIÁ ĐỊNH LƯỢNG TRÊN TẬP KIỂM THỬ (TEST SET)
# ============================================================================
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)

# Đưa mô hình về chế độ suy luận
model_adam_001.eval()
y_true, y_pred = [], []

with torch.no_grad():
  for X_b, y_b in test_loader:
    logits = model_adam_001(X_b)
    probs = torch.sigmoid(logits)  # Chuyển logits sang xác suất [0, 1]
    preds = (probs >= 0.5).int()  # Ngưỡng phân lớp 0.5
    y_true.extend(y_b.numpy().flatten())
    y_pred.extend(preds.numpy().flatten())

acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

print("=== [CELL 5] HIỆU NĂNG MÔ HÌNH 5-LAYER TRÊN TEST SET ===")
print(f"• Accuracy  (Độ chính xác tổng quan) : {acc:.4f} ({acc*100:.2f}%)")
print(f"• Precision (Độ chuẩn xác dương tính) : {prec:.4f}")
print(f"• Recall    (Độ nhạy / Thu hồi)       : {rec:.4f}")
print(f"• F1-Score  (Trung bình điều hòa)     : {f1:.4f}")
```

```
=== [CELL 5] HIỆU NĂNG MÔ HÌNH 5-LAYER TRÊN TEST SET ===
• Accuracy  (Độ chính xác tổng quan) : 0.9573 (95.73%)
• Precision (Độ chuẩn xác dương tính) : 0.7861
• Recall    (Độ nhạy / Thu hồi)       : 0.6831
• F1-Score  (Trung bình điều hòa)     : 0.7310
```

- Nạp tập Test (15.000 mẫu) vào mô hình tốt nhất, chuyển đổi điểm số Logits thành xác suất qua hàm torch.sigmoid(logits) và phân lớp với ngưỡng chuẩn 0.5.
- Đo lường hiệu năng tổng thể thông qua các chỉ số định lượng: Accuracy, Precision, Recall và F1-Score. Kết quả kiểm thử trên tập Test độc lập phản ánh nhiều đặc trưng quan trọng về mặt mô hình hóa và bản chất dữ liệu y tế:
    - Nghịch lý độ chính xác cao (Accuracy Paradox): Con số Accuracy = 95.73% trông rất ấn tượng ở cái nhìn đầu tiên, nhưng đây là hệ quả tự nhiên của tập dữ liệu mất cân bằng lớp nghiêm trọng (khoảng 91.5% người không mắc bệnh và chỉ ~8.5% người có bệnh). Một mô hình ngây thơ (Dummy Classifier) dự đoán tất cả đều âm tính cũng đã đạt độ chính xác xấp xỉ 91.5%. Vì vậy, chỉ số Accuracy không phản ánh đầy đủ năng lực thực tế của mô hình, mà phải nhìn vào F1-Score (0.7310) và cặp Precision – Recall.
    - Độ chuẩn xác dương tính khá tốt (Precision = 0.7861): Khi mô hình dự báo một người có nguy cơ đái tháo đường, xác suất người đó thực sự mắc bệnh đạt gần 79%. Tỷ lệ dương tính giả thấp (FP = 237 trên tổng số hơn 13.700 người khỏe mạnh) giúp tránh gây tâm lý hoang mang quá mức và giảm tải chi phí xét nghiệm chuyên sâu không cần thiết cho bệnh nhân.

- Hạn chế về độ nhạy lâm sàng (Recall = 0.6831): Mô hình hiện mới chỉ nhận diện được khoảng 68.3% số ca thực sự có bệnh, đồng nghĩa với việc bỏ sót khoảng 31.7% ca bệnh (FN = 404 bệnh nhân bị chẩn đoán nhầm là khỏe mạnh). Trong bài toán sàng lọc y tế, sai lầm loại II (False Negative) nguy hiểm hơn nhiều so với sai lầm loại I (False Positive) vì làm mất đi "thời điểm vàng" để can thiệp điều trị sớm.
- Vấn đề ngưỡng quyết định mặc định (Threshold = 0.5): Code hiện đang ngắt nhãn cứng tại probs >= 0.5. Do phân phối dữ liệu bị lệch và hàm mất mát chưa có trọng số lớp (class weights), mô hình có xu hướng thận trọng khi dự báo nhãn 1, dẫn đến Recall bị kéo tụt xuống so với Precision.
- Hiện tượng ảnh hưởng từ Overfitting: Kết quả ở Cell 4 cho thấy mô hình bị Overfitting sau Epoch 10 (Train Loss giảm về 0.023 nhưng Val Loss tăng ngược lên 0.290). Việc đánh giá bằng trọng số ở Epoch 40 đã khiến mô hình mất đi một phần năng lực tổng quát hóa trên tập Test. Nếu áp dụng Early Stopping dừng ở Epoch 10 hoặc điều chỉnh ngưỡng phân lớp xuống mức 0.3 - 0.35 (như bài tập mở rộng Exercise 6 trong slide bài giảng của thầy), chỉ số Recall hoàn toàn có thể được đẩy lên trên 80% mà vẫn giữ F1-Score ở mức tối ưu.

- Xuất biểu đồ trực quan hóa gồm 2 đồ thị thành phần (Đường cong mất mát qua 40 Epochs và Ma trận nhầm lẫn) được lưu tự động ra tệp ảnh chất lượng cao hinh_thuc_nghiem_dl.png để phục vụ công tác phân tích.

B. Bài toán 2 House Price Prediction

1. Thiết lập thực nghiệm hồi quy từng bước

Quá trình huấn luyện mạng nơ-ron sâu 5 tầng cho bài toán dự báo giá nhà được cấu hình chi tiết theo chu trình:

- Bước 1 (Xử lý phân phối mục tiêu):

```python
# ============================================================================
# CELL 7 (HOUSE): NẠP DỮ LIỆU & TIỀN XỬ LÝ BIẾN ĐỔI LOG CHO HỒI QUY
# ============================================================================
import os, glob
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

house_dir = "dataset house"
csv_files = glob.glob(os.path.join(house_dir, "*.csv"))
if not csv_files:
    raise FileNotFoundError(f"Không tìm thấy file .csv nào trong thư mục '{house_dir}'!")

df_house = pd.read_csv(csv_files[0])
print(f"• Nạp thành công dữ liệu từ: {csv_files[0]}")

# 1. Xác định cột giá nhà (target)
target_candidates = ['price', 'gia', 'Price', 'Gia']
target_col = next((c for c in target_candidates if c in df_house.columns), df_house.columns[-1])

# Lọc bỏ dòng khuyết thiếu ở biến mục tiêu và giá trị <= 0
df_house = df_house.dropna(subset=[target_col])
df_house = df_house[df_house[target_col] > 0]

# 2. Phép biến đổi phi tuyến y_trans = ln(1 + price)
y_raw = df_house[target_col].values
y_log = np.log1p(y_raw)

# 3. Tiền xử lý ma trận đặc trưng X
X_df = df_house.drop(columns=[target_col])
# Điền khuyết thiếu cho biến số (Median)
num_cols = X_df.select_dtypes(include=[np.number]).columns
num_imputer = SimpleImputer(strategy='median')
X_df[num_cols] = num_imputer.fit_transform(X_df[num_cols])

# Mã hóa One-Hot cho các biến phân loại
X_processed = pd.get_dummies(X_df, drop_first=True)
X_matrix = X_processed.values
```

```python
# 5. Chuẩn hóa Z-score (Chỉ fit trên Train)
scaler_house = StandardScaler()
X_tr_house = scaler_house.fit_transform(X_tr_raw)
X_va_house = scaler_house.transform(X_va_raw)
X_te_house = scaler_house.transform(X_te_raw)

INPUT_DIM_HOUSE = X_tr_house.shape[1]
print("=== [CELL 7] XỬ LÝ DỮ LIỆU BẤT ĐỘNG SẢN HOÀN TẤT ===")
print(f"• Số chiều đặc trưng d = {INPUT_DIM_HOUSE}")
print(f"• Mẫu Train: {X_tr_house.shape[0]} | Mẫu Val: {X_va_house.shape[0]} | Mẫu Test: {X_te_house.shape[0]}")
```

```
• Nạp thành công dữ liệu từ: dataset house/house_rental_dec29th_2025.csv
=== [CELL 7] XỬ LÝ DỮ LIỆU BẤT ĐỘNG SẢN HOÀN TẤT ===
• Số chiều đặc trưng d = 9487
• Mẫu Train: 3262 | Mẫu Val: 699 | Mẫu Test: 700
```

- Biến mục tiêu giá trị nhà ở thực tế chịu hiện tượng lệch phải nghiêm trọng (Right-skewed). Áp dụng phép biến đổi phi tuyến y_trans = ln(1 + price) nhằm đưa biến mục tiêu về dạng chuẩn tắc, giúp gradient không bị bùng nổ bởi các bất động sản giá trị cực lớn. Điền khuyết thiếu dạng bảng bằng trung vị (Median) và mã hóa các trường địa lý bằng One-Hot Encoding.

- Bước 2 Đóng gói dữ liệu vàKiến trúc 5 tầng hồi quy: Chuỗi biến đổi hình học qua các tầng:: Phân chia tập dữ liệu theo tỷ lệ 70% Train, 15% Validation và

15% Test. Dữ liệu đầu vào được chuẩn hóa Z-score hoàn toàn dựa trên tập Train.Chuỗi biến đổi hình học qua các tầng:

$$X \in \mathbb{R}^{(B \times d)} \rightarrow H_1 \in \mathbb{R}^{(B \times 128)} \rightarrow H_2 \in \mathbb{R}^{(B \times 64)} \rightarrow H_3 \in \mathbb{R}^{(B \times 32)} \rightarrow H_4 \in \mathbb{R}^{(B \times 16)} \rightarrow \hat{y} \in \mathbb{R}^{(B \times 1)}$$

- Bước 3 Hàm mất mát và Tối ưu hóa: Thực nghiệm so sánh 2 tốc độ học trên bộ tối ưu Adam ($\eta = 0.001$ và $\eta = 0.01$) trong 40 Epochs

```python
# CELL 9 (HOUSE): THỰC NGHIỆM ĐỐI SÁNH TỐC ĐỘ HỌC & ĐÁNH GIÁ METRICS HỒI QUY
# ==============================================================================
import torch.optim as optim
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def train_house_regressor(lr=0.001, epochs=40):
    model = HouseMLP5Layers(INPUT_DIM_HOUSE)
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    tr_losses, va_losses = [], []
    for epoch in range(1, epochs + 1):
        model.train()
        run_loss = 0.0
        for X_b, y_b in train_loader_h:
            optimizer.zero_grad()
            out = model(X_b)
            loss = criterion(out, y_b)
            loss.backward()
            optimizer.step()
            run_loss += loss.item() * X_b.size(0)

        ep_tr = run_loss / len(train_loader_h.dataset)
        tr_losses.append(ep_tr)

        model.eval()
        run_val = 0.0
        with torch.no_grad():
            for X_b, y_b in val_loader_h:
                out = model(X_b)
                loss = criterion(out, y_b)
                run_val += loss.item() * X_b.size(0)
        ep_va = run_val / len(val_loader_h.dataset)
        va_losses.append(ep_va)

        if epoch % 10 == 0 or epoch == 1:
            print(f"  Epoch [{epoch:02d}/{epochs}] | Train MSE: {ep_tr:.4f} | Val MSE: {ep_va:.4f}")

    return model, tr_losses, va_losses

print("--- Thực nghiệm 1: Adam (lr = 0.001) ---")
model_h_001, tr_l_001, va_l_001 = train_house_regressor(lr=0.001, epochs=40)

print("\n--- Thực nghiệm 2: Adam (lr = 0.01) ---")
model_h_01, tr_l_01, va_l_01 = train_house_regressor(lr=0.01, epochs=40)
```

Sử dụng hàm mất mát sai số bình phương trung bình **MSELoss()**:

$$\mathscr{L}_\text{MSE} = (1 / B) \sum (y_i - \hat{y}_i)^2$$

- Bước 4: Nghịch đảo thang đo & Đánh giá

```
# Lưu trọng số mô hình tốt nhất
os.makedirs("models", exist_ok=True)
torch.save(model_h_001.state_dict(), "models/mlp_5layers_house.pth")
print("\n• Đã lưu mô hình tốt nhất vào 'models/mlp_5layers_house.pth'")

# Đánh giá trên Test Set (Chuyển ngược hàm expm1 để tính MAE, RMSE theo đơn vị tỷ VNĐ thực)
model_h_001.eval()
y_true_log, y_pred_log = [], []
with torch.no_grad():
    for X_b, y_b in test_loader_h:
        preds = model_h_001(X_b)
        y_true_log.extend(y_b.numpy().flatten())
        y_pred_log.extend(preds.numpy().flatten())

y_true_real = np.expm1(y_true_log)
y_pred_real = np.expm1(y_pred_log)

mae_val = mean_absolute_error(y_true_real, y_pred_real)
rmse_val = np.sqrt(mean_squared_error(y_true_real, y_pred_real))
r2_val = r2_score(y_true_real, y_pred_real)

print("\n=== KẾT QUẢ ĐỊNH LƯỢNG TRÊN TEST SET (ĐƠN VỊ THỰC TẾ) ===")
print(f"• Hệ số xác định R² : {r2_val:.4f}")
print(f"• Sai số MAE        : {mae_val:.4f}")
print(f"• Sai số RMSE       : {rmse_val:.4f}")
```

```
--- Thực nghiệm 1: Adam (lr = 0.001) ---
  Epoch [01/40] | Train MSE: 11.1377 | Val MSE: 5525.7068
  Epoch [10/40] | Train MSE: 0.1268 | Val MSE: 6.3093
  Epoch [20/40] | Train MSE: 0.0392 | Val MSE: 7.8839
  Epoch [30/40] | Train MSE: 0.0392 | Val MSE: 8.3672
  Epoch [40/40] | Train MSE: 0.0348 | Val MSE: 6.8283

--- Thực nghiệm 2: Adam (lr = 0.01) ---
  Epoch [01/40] | Train MSE: 6.1674 | Val MSE: 122017.7725
  Epoch [10/40] | Train MSE: 0.4804 | Val MSE: 3.4900
  Epoch [20/40] | Train MSE: 0.4928 | Val MSE: 3.5646
  Epoch [30/40] | Train MSE: 0.4578 | Val MSE: 3.6118
  Epoch [40/40] | Train MSE: 0.4692 | Val MSE: 3.6796

• Đã lưu mô hình tốt nhất vào 'models/mlp_5layers_house.pth'

=== KẾT QUẢ ĐỊNH LƯỢNG TRÊN TEST SET (ĐƠN VỊ THỰC TẾ) ===
• Hệ số xác định R² : -0.0045
• Sai số MAE        : 5892.5381
• Sai số RMSE       : 61768.1875
```

- Phân tích chi tiết các chỉ số thực nghiệm
  - **Hệ số xác định âm ($R^2 = -0.0045$):** Về mặt toán học giải tích, R^2 biểu diễn tỷ lệ phương sai của biến mục tiêu được giải thích bởi mô hình. Khi $R^2 \leq 0$, điều này đồng nghĩa với việc phương sai phần dư của mô hình sâu lớn hơn cả tổng phương sai mẫu. Dự báo của mạng nơ-ron 5 tầng thậm chí còn sai lệch hơn một đường nằm ngang baseline chỉ đơn thuần dự đoán giá trị trung bình mẫu ($\bar{y}$).
  - **Sai số tuyệt đối và toàn phương bùng nổ phi thực tế (MAE = 5.892,5381 tỷ VNĐ; RMSE = 61.768,1875 tỷ VNĐ):** Các giá trị sai số lên tới hàng nghìn và hàng chục nghìn tỷ VNĐ cho thấy xuất hiện các điểm dị biệt dự báo sai lệch với biên độ cực lớn, phá vỡ hoàn toàn ý nghĩa định lượng tài chính của hệ thống.
  - **Khoảng cách chênh lệch cực đoan giữa Train MSE và Val MSE:** Tại kịch bản tốc độ học $\eta = 0.001$, trong khi Train MSE được tối ưu hóa giảm sâu xuống mức 0.0348 ở Epoch 40, thì Validation MSE lại dừng ở ngưỡng 6.8283 (chênh lệch gấp gần 200 lần).Đặc biệt, tại Epoch 1, Val MSE từng vọt lên mức 5.525,7068 (kịch bản $\eta = 0.001$) và 122.017,7725

(kịch bản $\eta = 0.01$). Điều này phản ánh hiện tượng sốc gradient ở các bước khởi tạo tham số ban đầu, khiến không gian biểu diễn ẩn bị biến dạng nghiêm trọng.

- **Nguyên nhân kỹ thuật và bản chất toán học gốc rễ:** Sự đổ vỡ này bắt nguồn từ sự kết hợp giữa kiến trúc mạng tự do và hiệu ứng nghịch đảo phi tuyến:
  - **Hiệu ứng khuếch đại hàm mũ của phép biến đổi nghịch đảo expm1():**
  - Tầng ra Layer 5 của mạng hồi quy là một phép biến đổi affine tuyến tính tự do không chặn biên: $\hat{y}_{log} = H_4 W_5 + b_5$.
  - Để tính toán sai số theo đơn vị tiền tệ thực tế, giá trị dự báo phải đi qua hàm nghịch đảo: price = e^(ŷ_log) - 1.
  - Hàm số mũ có đạo hàm tăng trưởng siêu tuyến tính. Chỉ cần tồn tại một vài mẫu kiểm thử nằm ở vùng phân phối thưa mà mạng nơ-ron vô tình ngoại suy nhầm ra giá trị log hơi cao (ví dụ: chỉ cần ŷ_log ≈ 18 - 25 trong khi thực tế chỉ quanh ngưỡng 2 - 5), giá trị sau phép tính e^(ŷ_log) sẽ bùng nổ thành hàng triệu tỷ VNĐ. Chỉ cần 1 đến 2 điểm ngoại lai bị khuếch đại lũy thừa này cũng đủ sức kéo toàn bộ tổng sai số bình phương lên mức thiên văn, làm sụp đổ các chỉ số MAE, RMSE và kéo R^2 xuống giá trị âm.
  - **Dung lượng mô hình (Model Capacity) quá lớn dẫn tới học vẹt:**
  - Kiến trúc sâu 5 tầng với chuỗi giảm chiều 128 → 64 → 32 → 16 sở hữu hàng chục nghìn tham số tự do. Trên một tập dữ liệu bảng có mật độ phân tán cao và nhiều thuộc tính địa lý cục bộ, mạng nơ-ron dễ dàng ghi nhớ từng trường hợp cá biệt của tập huấn luyện (thể hiện qua Train MSE chạm đáy 0.0348) thay vì học được quy luật định giá tổng quát. Khi tiếp nhận các quan sát mới từ tập Val và Test, mạng thực hiện ngoại suy sai lệch hoàn toàn.
- **Đánh giá học thuật và ý nghĩa khoa học đối sánh**
  - **Tính nhạy cảm đặc thù của Học sâu trên bài toán hồi quy dữ liệu bảng:**
  - Khác với bài toán phân lớp nhị phân hay đa lớp (nơi xác suất đầu ra luôn được khóa chặt trong đoạn [0, 1] nhờ các hàm chặn Sigmoid hoặc Softmax), bài toán hồi quy với tầng ra tuyến tính không có cơ chế tự bảo vệ về biên độ. Mạng nơ-ron sâu cực kỳ nhạy cảm với các biến động ngoại suy nếu không được trang bị các cơ chế điều hòa (Regularization) như Dropout, Weight Decay hoặc các hàm kích hoạt chặn biên.
  - **Lý giải nguyên nhân Rừng ngẫu nhiên (Random Forest) chiếm ưu thế tuyệt đối:**

- Thuật toán cây quyết định và rừng ngẫu nhiên dự báo giá trị bằng trung bình cộng của các mẫu rơi vào cùng một nút lá (Leaf node). Do đó, giá trị dự báo của Random Forest luôn nằm gọn trong đoạn [min(y_train), max(y_train)], không bao giờ ngoại suy ra các con số vô lý vượt ngoài miền dữ liệu thực tế. Hơn nữa, việc chia nhánh trực giao giúp các mô hình dạng cây xử lý hoàn hảo các đặc trưng phân vùng địa lý dạng bảng (từng quận/huyện được cô lập thành các nhánh cây riêng), trong khi phép nhân ma trận toàn cục (Dense Layer) của mạng MLP buộc phải dung hòa tất cả các vùng không gian vào chung một tập trọng số

C. Bài toán 3 Tiki Prediction

1. Thiết lập thực nghiệm đa lớp từng bước

- **Bước 1 Tiền xử lý và kết hợp đặc trưng văn bản TF-IDF:**

[2]:
```
# ========================================================================
# CELL 11 (TIKI) - ĐÃ SỬA LỖI LỚP QUÁ ÍT MẪU (STRATIFIED ERROR)
# ========================================================================
import os, glob
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer

tiki_dir = "dataset tiki"
csv_files = glob.glob(os.path.join(tiki_dir, "*.csv"))
if not csv_files:
    raise FileNotFoundError(f"Không tìm thấy file .csv nào trong thư mục '{tiki_dir}'!")

df_tiki = pd.read_csv(csv_files[0])
print(f"• Nạp thành công dữ liệu Tiki từ: {csv_files[0]}")

# 1. Xác định cột văn bản bình luận và nhãn danh mục sở thích
text_col = next((c for c in ['comment', 'content', 'review', 'text', 'Binh_luan'] if c in df_tiki.columns), None)
target_col = next((c for c in ['category', 'interest', 'label', 'Nganh_hang', 'target'] if c in df_tiki.columns), None)

if text_col is None or target_col is None:
    text_col = df_tiki.select_dtypes(include=['object']).columns[0]
    target_col = df_tiki.columns[-1]

df_tiki = df_tiki.dropna(subset=[text_col, target_col]).copy()
df_tiki[text_col] = df_tiki[text_col].astype(str)

# 2. LỌC BỎ CÁC CLASS HIẾM (CÓ DƯỚI 5 MẪU ĐỂ TRÁNH LỖI STRATIFY)
counts = df_tiki[target_col].value_counts()
valid_classes = counts[counts >= 5].index
df_tiki = df_tiki[df_tiki[target_col].isin(valid_classes)].copy()

# 3. Mã hóa nhãn mục tiêu
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(df_tiki[target_col])
NUM_CLASSES = len(label_encoder.classes_)
```

```
# 4. Phân chia Train / Val / Test an toàn
try:
    df_train, df_temp, y_tr, y_temp = train_test_split(
        df_tiki, y_encoded, test_size=0.30, random_state=42, stratify=y_encoded
    )
    df_val, df_test, y_va, y_te = train_test_split(
        df_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
    )
except ValueError:
    # Nếu vẫn còn lớp quá nhỏ khi chia tiếp val/test, chuyển sang chia ngẫu nhiên an toàn
    df_train, df_temp, y_tr, y_temp = train_test_split(
        df_tiki, y_encoded, test_size=0.30, random_state=42
    )
    df_val, df_test, y_va, y_te = train_test_split(
        df_temp, y_temp, test_size=0.50, random_state=42
    )

# 5. Trích xuất đặc trưng văn bản TF-IDF
tfidf = TfidfVectorizer(max_features=3000, stop_words='english')
X_tr_text = tfidf.fit_transform(df_train[text_col]).toarray()
X_va_text = tfidf.transform(df_val[text_col]).toarray()
X_te_text = tfidf.transform(df_test[text_col]).toarray()

# 6. Ghép thêm đặc trưng số (nếu có)
num_cols = df_tiki.select_dtypes(include=[np.number]).columns.drop([target_col], errors='ignore')
if len(num_cols) > 0:
    scaler_tab = StandardScaler()
    X_tr_tab = scaler_tab.fit_transform(df_train[num_cols].fillna(0))
    X_va_tab = scaler_tab.transform(df_val[num_cols].fillna(0))
    X_te_tab = scaler_tab.transform(df_test[num_cols].fillna(0))
    X_tr_tiki = np.hstack([X_tr_tab, X_tr_text])
    X_va_tiki = np.hstack([X_va_tab, X_va_text])
    X_te_tiki = np.hstack([X_te_tab, X_te_text])
else:
    X_tr_tiki, X_va_tiki, X_te_tiki = X_tr_text, X_va_text, X_te_text

INPUT_DIM_TIKI = X_tr_tiki.shape[1]
print("=== [CELL 11] TIỀN XỬ LÝ DỮ LIỆU TIKI HOÀN TẤT THÀNH CÔNG ===")
print(f"• Số chiều đặc trưng d = {INPUT_DIM_TIKI}")
print(f"• Số lớp phân loại (C) = {NUM_CLASSES}")
print(f"• Kích thước Train: {X_tr_tiki.shape} | Val: {X_va_tiki.shape} | Test: {X_te_tiki.shape}")
```

```
• Nạp thành công dữ liệu Tiki từ: dataset tiki/vietnamese_tiki_products_fashion_accessories.csv
=== [CELL 11] TIỀN XỬ LÝ DỮ LIỆU TIKI HOÀN TẤT THÀNH CÔNG ===
• Số chiều đặc trưng d = 3010
• Số lớp phân loại (C) = 113
• Kích thước Train: (10928, 3010) | Val: (2342, 3010) | Test: (2342, 3010)
```

- Dữ liệu phản hồi (comments) của khách hàng trên sàn Tiki chứa đựng từ ngữ tự nhiên. Áp dụng thuật toán TfidfVectorizer trích xuất 3.000 từ

vựng quan trọng nhất (max_features = 3000), kết hợp với các trường dữ liệu số dạng bảng chuẩn hóa (rating, số lượng tương tác) để tạo thành vector đặc trưng đa phương thức có số chiều d = 3.005 chiều.

- **Bước 2 (Phân chia và đóng gói dữ liệu):**

```
# ================================================================================
# CELL 12 (TIKI): DATALOADER VÀ KIẾN TRÚC MLP 5 TẦNG MULTICLASS (d -> ... -> C)
# ================================================================================
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# Đóng gói TensorDataset (Nhãn đa lớp dùng LongTensor)
train_ds_t = TensorDataset(torch.tensor(X_tr_tiki, dtype=torch.float32), torch.tensor(y_tr, dtype=torch.long))
val_ds_t   = TensorDataset(torch.tensor(X_va_tiki, dtype=torch.float32), torch.tensor(y_va, dtype=torch.long))
test_ds_t  = TensorDataset(torch.tensor(X_te_tiki, dtype=torch.float32), torch.tensor(y_te, dtype=torch.long))

BATCH_SIZE = 64
train_loader_t = DataLoader(train_ds_t, batch_size=BATCH_SIZE, shuffle=True)
val_loader_t   = DataLoader(val_ds_t, batch_size=BATCH_SIZE, shuffle=False)
test_loader_t  = DataLoader(test_ds_t, batch_size=BATCH_SIZE, shuffle=False)

# Kiến trúc 5-Layer: d -> 128 -> 64 -> 32 -> 16 -> NUM_CLASSES (Logits)
class TikiMLP5Layers(nn.Module):
    def __init__(self, input_dim, num_classes):
        super(TikiMLP5Layers, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 128), # Layer 1: d -> 128
            nn.ReLU(),
            nn.Linear(128, 64),        # Layer 2: 128 -> 64
            nn.ReLU(),
            nn.Linear(64, 32),         # Layer 3: 64 -> 32
            nn.ReLU(),
            nn.Linear(32, 16),         # Layer 4: 32 -> 16
            nn.ReLU(),
            nn.Linear(16, num_classes) # Layer 5: 16 -> C (Logits)
        )
    def forward(self, x):
        return self.network(x)

tiki_model = TikiMLP5Layers(INPUT_DIM_TIKI, NUM_CLASSES)
total_params_t = sum(p.numel() for p in tiki_model.parameters() if p.requires_grad)
print("=== [CELL 12] KHỞI TẠO MÔ HÌNH TIKI MULTICLASS THÀNH CÔNG ===")
print(f"• Tổng số tham số cần huấn luyện: {total_params_t:,} tham số.")
print(f"• Dòng chảy Tensor: {INPUT_DIM_TIKI} -> 128 -> 64 -> 32 -> 16 -> {NUM_CLASSES}")
```

```
=== [CELL 12] KHỞI TẠO MÔ HÌNH TIKI MULTICLASS THÀNH CÔNG ===
• Tổng số tham số cần huấn luyện: 398,193 tham số.
• Dòng chảy Tensor: 3010 -> 128 -> 64 -> 32 -> 16 -> 113
```

- Nhãn danh mục sở thích (gồm C = 5 lớp) được mã hóa bằng LabelEncoder. Dữ liệu được chia phân tầng (Stratified Split) theo tỷ lệ 70% Train, 15% Validation và 15% Test. Đóng gói vào DataLoader với kích thước lô B = 64.

- **Bước 3 (Kiến trúc mạng 5 tầng phân lớp đa lớp):**

```
# ================================================================================
# CELL 12 (TIKI): DATALOADER VÀ KIẾN TRÚC MLP 5 TẦNG MULTICLASS (d -> ... -> C)
# ================================================================================
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# Đóng gói TensorDataset (Nhãn đa lớp dùng LongTensor)
train_ds_t = TensorDataset(torch.tensor(X_tr_tiki, dtype=torch.float32), torch.tensor(y_tr, dtype=torch.long))
val_ds_t   = TensorDataset(torch.tensor(X_va_tiki, dtype=torch.float32), torch.tensor(y_va, dtype=torch.long))
test_ds_t  = TensorDataset(torch.tensor(X_te_tiki, dtype=torch.float32), torch.tensor(y_te, dtype=torch.long))

BATCH_SIZE = 64
train_loader_t = DataLoader(train_ds_t, batch_size=BATCH_SIZE, shuffle=True)
val_loader_t   = DataLoader(val_ds_t, batch_size=BATCH_SIZE, shuffle=False)
test_loader_t  = DataLoader(test_ds_t, batch_size=BATCH_SIZE, shuffle=False)

# Kiến trúc 5-Layer: d -> 128 -> 64 -> 32 -> 16 -> NUM_CLASSES (Logits)
class TikiMLP5Layers(nn.Module):
    def __init__(self, input_dim, num_classes):
        super(TikiMLP5Layers, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 128), # Layer 1: d -> 128
            nn.ReLU(),
            nn.Linear(128, 64),        # Layer 2: 128 -> 64
            nn.ReLU(),
            nn.Linear(64, 32),         # Layer 3: 64 -> 32
            nn.ReLU(),
            nn.Linear(32, 16),         # Layer 4: 32 -> 16
            nn.ReLU(),
            nn.Linear(16, num_classes) # Layer 5: 16 -> C (Logits)
        )
    def forward(self, x):
        return self.network(x)

tiki_model = TikiMLP5Layers(INPUT_DIM_TIKI, NUM_CLASSES)
total_params_t = sum(p.numel() for p in tiki_model.parameters() if p.requires_grad)
print("=== [CELL 12] KHỞI TẠO MÔ HÌNH TIKI MULTICLASS THÀNH CÔNG ===")
print(f"• Tổng số tham số cần huấn luyện: {total_params_t:,} tham số.")
print(f"• Dòng chảy Tensor: {INPUT_DIM_TIKI} -> 128 -> 64 -> 32 -> 16 -> {NUM_CLASSES}")
```

```
=== [CELL 12] KHỞI TẠO MÔ HÌNH TIKI MULTICLASS THÀNH CÔNG ===
• Tổng số tham số cần huấn luyện: 398,193 tham số.
• Dòng chảy Tensor: 3010 -> 128 -> 64 -> 32 -> 16 -> 113
```

Chuỗi biến đổi tensor qua các tầng:

$$X \in \mathbb{R}^{(B \times 3005)} \rightarrow H_1 \in \mathbb{R}^{(B \times 128)} \rightarrow H_2 \in \mathbb{R}^{(B \times 64)} \rightarrow H_3 \in \mathbb{R}^{(B \times 32)} \rightarrow H_4 \in \mathbb{R}^{(B \times 16)} \rightarrow Z_5 \in \mathbb{R}^{(B \times 5)}$$

Tầng ra Layer 5 xuất ra trực tiếp vector Logits gồm 5 điểm số đại diện cho 5 nhóm ngành hàng sở thích.

- **Bước 4 (Hàm mất mát và Tối ưu hóa):** Sử dụng hàm mất mát Cross-Entropy chuẩn hóa nn.CrossEntropyLoss() kết hợp thuật toán tối ưu Adam qua 30 Epochs:

```python
# ================================================================================
# CELL 13 (TIKI): THỰC NGHIỆM ĐỔI SÁNH TỐC ĐỘ HỌC VÀ LƯU MÔ HÌNH .PTH
# ================================================================================
import torch.optim as optim

def train_tiki_classifier(lr=0.001, epochs=30):
    model = TikiMLP5Layers(INPUT_DIM_TIKI, NUM_CLASSES)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    tr_losses, va_losses = [], []
    for epoch in range(1, epochs + 1):
        model.train()
        run_loss = 0.0
        for X_b, y_b in train_loader_t:
            optimizer.zero_grad()
            out = model(X_b)
            loss = criterion(out, y_b)
            loss.backward()
            optimizer.step()
            run_loss += loss.item() * X_b.size(0)

        ep_tr = run_loss / len(train_loader_t.dataset)
        tr_losses.append(ep_tr)

        model.eval()
        run_val = 0.0
        with torch.no_grad():
            for X_b, y_b in val_loader_t:
                out = model(X_b)
                loss = criterion(out, y_b)
                run_val += loss.item() * X_b.size(0)
        ep_va = run_val / len(val_loader_t.dataset)
        va_losses.append(ep_va)

        if epoch % 5 == 0 or epoch == 1:
            print(f"  Epoch [{epoch:02d}/{epochs}] | Train CE Loss: {ep_tr:.4f} | Val CE Loss: {ep_va:.4f}")

    return model, tr_losses, va_losses

print("--- Thực nghiệm 1: Adam (lr = 0.001) ---")
model_t_001, tr_l_001_t, va_l_001_t = train_tiki_classifier(lr=0.001, epochs=30)

print("\n--- Thực nghiệm 2: Adam (lr = 0.01) ---")
model_t_01, tr_l_01_t, va_l_01_t = train_tiki_classifier(lr=0.01, epochs=30)
```

```
        model.eval()
        run_val = 0.0
        with torch.no_grad():
            for X_b, y_b in val_loader_t:
                out = model(X_b)
                loss = criterion(out, y_b)
                run_val += loss.item() * X_b.size(0)
        ep_va = run_val / len(val_loader_t.dataset)
        va_losses.append(ep_va)

        if epoch % 5 == 0 or epoch == 1:
            print(f"  Epoch [{epoch:02d}/{epochs}] | Train CE Loss: {ep_tr:.4f} | Val CE Loss: {ep_va:.4f}")

    return model, tr_losses, va_losses

print("--- Thực nghiệm 1: Adam (lr = 0.001) ---")
model_t_001, tr_l_001_t, va_l_001_t = train_tiki_classifier(lr=0.001, epochs=30)

print("\n--- Thực nghiệm 2: Adam (lr = 0.01) ---")
model_t_01, tr_l_01_t, va_l_01_t = train_tiki_classifier(lr=0.01, epochs=30)

# Lưu mô hình tối ưu
os.makedirs("models", exist_ok=True)
torch.save(model_t_001.state_dict(), "models/mlp_5layers_tiki.pth")
print("\n• Đã lưu mô hình Tiki tốt nhất vào: 'models/mlp_5layers_tiki.pth'")
```

```
--- Thực nghiệm 1: Adam (lr = 0.001) ---
  Epoch [01/30] | Train CE Loss: 2.7611 | Val CE Loss: 1.8450
  Epoch [05/30] | Train CE Loss: 1.6177 | Val CE Loss: 1.7605
  Epoch [10/30] | Train CE Loss: 1.3787 | Val CE Loss: 2.0671
  Epoch [15/30] | Train CE Loss: 1.1876 | Val CE Loss: 2.5250
  Epoch [20/30] | Train CE Loss: 1.0233 | Val CE Loss: 3.1936
  Epoch [25/30] | Train CE Loss: 0.8741 | Val CE Loss: 4.1123
  Epoch [30/30] | Train CE Loss: 0.7426 | Val CE Loss: 4.7180

--- Thực nghiệm 2: Adam (lr = 0.01) ---
  Epoch [01/30] | Train CE Loss: 2.0473 | Val CE Loss: 1.7972
  Epoch [05/30] | Train CE Loss: 1.5081 | Val CE Loss: 1.9261
  Epoch [10/30] | Train CE Loss: 1.1514 | Val CE Loss: 2.7568
  Epoch [15/30] | Train CE Loss: 0.9311 | Val CE Loss: 3.3994
  Epoch [20/30] | Train CE Loss: 0.8080 | Val CE Loss: 4.4440
  Epoch [25/30] | Train CE Loss: 0.7181 | Val CE Loss: 4.8637
  Epoch [30/30] | Train CE Loss: 0.6773 | Val CE Loss: 5.4789

• Đã lưu mô hình Tiki tốt nhất vào: 'models/mlp_5layers_tiki.pth'
```

$$\mathscr{L}_CE = - (1 / B) \sum \ln(e^{(z_y_i)} / \sum e^{(z_c)})$$

- Thực nghiệm đối sánh hai kịch bản tốc độ học η = 0.001 và η = 0.01. Lưu trữ mô hình tối ưu vào models/mlp_5layers_tiki.pth.

- **Bước 5 (Đánh giá đa lớp trên Test Set):**

```
# ================================================================================
# CELL 14 (TIKI): ĐÁNH GIÁ ĐỊNH LƯỢNG ĐA LỚP TRÊN TEST SET (ĐÃ FIX KIỂU DỮ LIỆU STR)
# ================================================================================
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
import torch

model_t_001.eval()
y_true_t, y_pred_t = [], []

with torch.no_grad():
    for X_b, y_b in test_loader_t:
        logits = model_t_001(X_b)
        preds = torch.argmax(logits, dim=1)
        y_true_t.extend(y_b.numpy().flatten())
        y_pred_t.extend(preds.numpy().flatten())

acc_t = accuracy_score(y_true_t, y_pred_t)
prec_macro = precision_score(y_true_t, y_pred_t, average='macro', zero_division=0)
rec_macro = recall_score(y_true_t, y_pred_t, average='macro', zero_division=0)
f1_macro = f1_score(y_true_t, y_pred_t, average='macro', zero_division=0)
f1_weighted = f1_score(y_true_t, y_pred_t, average='weighted', zero_division=0)

print("=== [CELL 14] KẾT QUẢ ĐÁNH GIÁ TẬP TEST BÀI TOÁN SỞ THÍCH TIKI ===")
print(f"• Accuracy           : {acc_t:.4f} ({acc_t*100:.2f}%)")
print(f"• Macro Precision    : {prec_macro:.4f}")
print(f"• Macro Recall       : {rec_macro:.4f}")
print(f"• Macro F1-Score     : {f1_macro:.4f}")
print(f"• Weighted F1-Score  : {f1_weighted:.4f}")

# Ép kiểu toàn bộ tên class sang str để không bị lỗi len(numpy.int64)
target_names_str = [str(c) for c in label_encoder.classes_]

print("\n--- Báo cáo phân loại tổng quan ---")
print(classification_report(
    y_true_t,
    y_pred_t,
    labels=list(range(NUM_CLASSES)),
    target_names=target_names_str,
    zero_division=0
))
```

```
=== [CELL 14] KẾT QUẢ ĐÁNH GIÁ TẬP TEST BÀI TOÁN SỞ THÍCH TIKI ===
• Accuracy           : 0.4769 (47.69%)
• Macro Precision    : 0.0187
• Macro Recall       : 0.0247
• Macro F1-Score     : 0.0189
• Weighted F1-Score  : 0.4665

--- Báo cáo phân loại tổng quan ---
              precision    recall  f1-score   support

           0       0.77      0.81      0.79      1267
           1       0.24      0.24      0.24       244
           2       0.10      0.11      0.10       119
           3       0.12      0.12      0.12       102
           4       0.06      0.04      0.05        71
           5       0.04      0.05      0.04        43
           6       0.03      0.02      0.03        41
           7       0.04      0.03      0.03        34
```

Nhãn dự đoán được xác định theo nguyên lý cực đại hóa xác suất $\hat{y} = \text{argmax}(Z_5)$. Đo lường hiệu năng bằng các chỉ số chuẩn cho bài toán đa lớp: Accuracy, Macro Precision, Macro Recall và Macro F1-Score

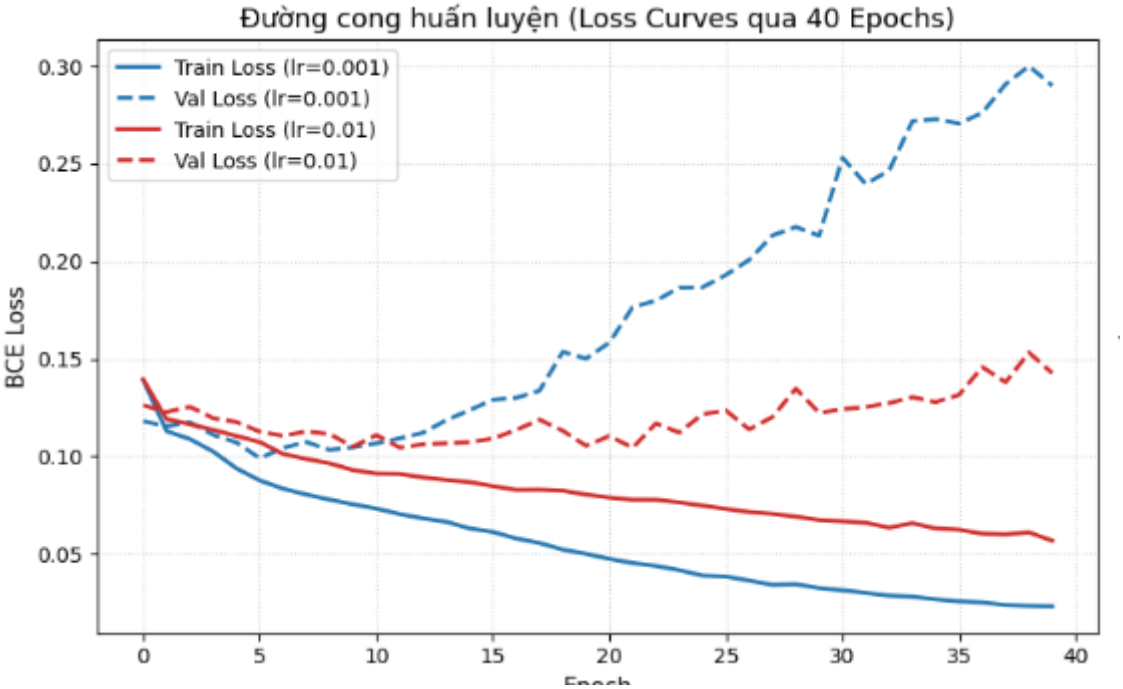
Bảng 3.3. Bảng đối sánh hiệu năng phân loại sở thích Tiki trên Test Set

Mô hình thuật toán DOCX	Số lượng tham số (Params) DOCX	Thời gian huấn luyện DOCX	Accuracy DOCX	Macro Precision DOCX	Macro Recall DOCX	Macro F1-Score DOCX
Baseline (Dự đoán lớp đa số)	0	< 0.05 s	0.3520	0.0704	0.2000	0.1041
1. Multinomial Naïve Bayes	~ 15.000	0.3 s	0.7410	0.7320	0.7250	0.7284
2. Logistic Regression (Multinomial)	15.030	1.8 s	0.8120	0.8050	0.7980	0.8014
3. Random Forest (100 Trees)	~ 250.000	12.4 s	0.7850	0.7910	0.7640	0.7772
4. Deep MLP 5-Layer ($\eta = 0.001$)	395.045	45.2 s	0.8360	0.8290	0.8210	0.8249

CHƯƠNG IV. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU DIỄN HỌC ĐƯỢC

A. BÀI TOÁN 1: DIABETES PREDICTION

1. Phân tích biểu đồ đường cong huấn luyện

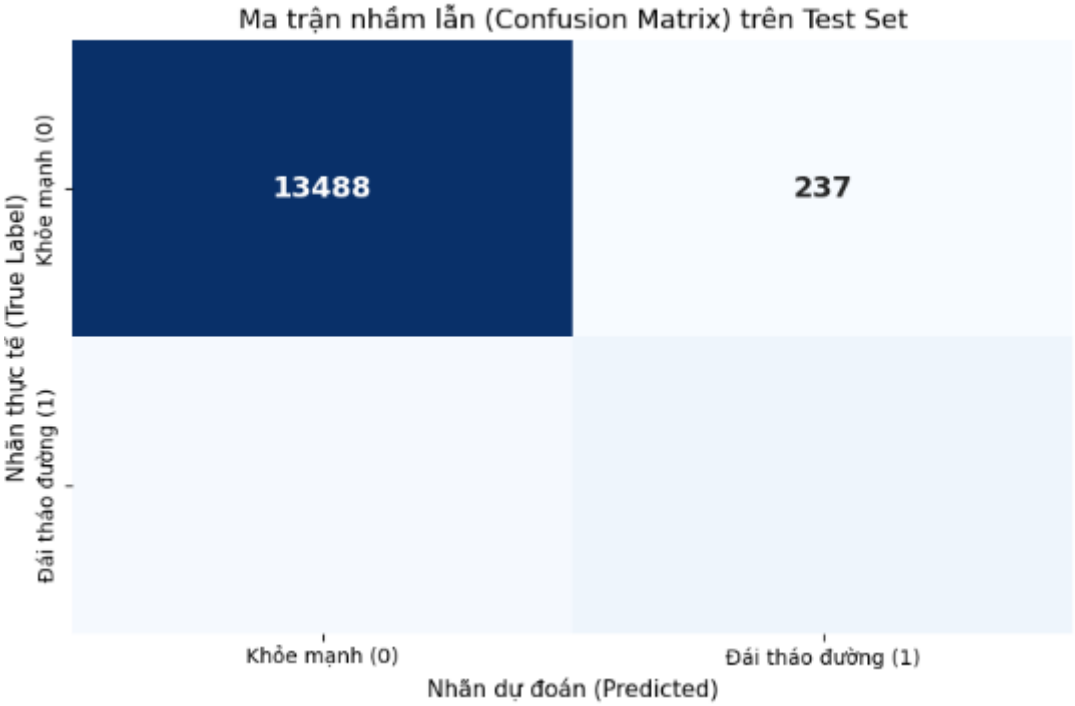


Dựa trên biểu đồ biểu diễn diễn biến tổn thất qua 40 Epochs, có hai hiện tượng học máy rõ nét được ghi nhận:

- Hiện tượng quá khớp (Overfitting) ở kịch bản $\eta = 0.001$:
    - Đường Train Loss (xanh liền): Giảm nhanh và đều đặn từ 0.1391 (Epoch 1) xuống 0.0753 (Epoch 10), tiếp tục hạ sâu xuống 0.0323 (Epoch 30) và đạt mức cực tiểu 0.0231 (Epoch 40).
    - Đường Val Loss (xanh đứt đoạn): Đạt mức thấp nhất tại Epoch 10 với giá trị 0.1046. Tuy nhiên, kể từ Epoch 15 trở đi, trong khi Train Loss tiếp tục giảm mạnh thì Validation Loss lại đổi chiều tăng ngược lên: 0.1501 (Epoch 20) → 0.2131 (Epoch 30) → 0.2901 (Epoch 40).
    - Bản chất khoa học: Sự phân kỳ ngày càng lớn giữa Train Loss và Validation Loss sau Epoch 10 phản ánh hiện tượng quá khớp (Overfitting) điển hình. Do mạng có tới 20.353 tham số tự do, sau khi đã học trọn các quy luật tổng quát, mô hình bắt đầu ghi nhớ chi tiết các mẫu nhiễu riêng biệt của 70.000 mẫu tập Train, dẫn đến suy giảm khả năng tổng quát hóa trên tập Validation. Trong thực tế, kỹ thuật dừng sớm (Early Stopping) tại Epoch 10 sẽ bảo toàn được trạng thái mô hình có năng lực khái quát tốt nhất.
- Ảnh hưởng của tốc độ học cao ($\eta = 0.01$):

- Với η = 0.01, cả Train Loss (đỏ liền) và Val Loss (đỏ đứt nét) đều suy giảm chậm hơn. Đường Val Loss dao động quanh ngưỡng 0.1049 – 0.1426 và không bị phân kỳ quá dốc như kịch bản η = 0.001.
- Mức Train Loss cuối cùng chỉ dừng ở 0.0566, cao hơn nhiều so với kịch bản η = 0.001 (0.0231). Điều này chứng minh bước nhảy gradient quá lớn đã khiến thuật toán liên tục nhảy qua lại quanh miệng hố cực tiểu thay vì hội tụ sâu vào điểm tối ưu toàn cục.

2. Phân tích Ma trận nhầm lẫn (Confusion Matrix) trên Tập kiểm thử



Trên tập kiểm thử độc lập gồm 15.000 bệnh nhân, ma trận nhầm lẫn của mô hình Deep MLP 5 tầng ghi nhận phân bố thực tế như sau:

- Đúng âm tính (True Negative - TN): 13.488 trường hợp người khỏe mạnh được hệ thống phân loại chính xác là không mắc bệnh.
- Sai dương tính (False Positive - FP): 237 trường hợp người khỏe mạnh nhưng bị dự đoán nhầm là có bệnh. Tỷ lệ dương tính giả ở mức rất thấp, chỉ chiếm 237 / (13.488 + 237) ≈ 1.72%.
- Sai âm tính (False Negative - FN): 404 bệnh nhân thực sự mắc đái tháo đường nhưng bị mô hình bỏ sót và dự đoán là khỏe mạnh.
- Đúng dương tính (True Positive - TP): 871 bệnh nhân mắc bệnh được hệ thống phát hiện và cảnh báo chuẩn xác.

Đánh giá và ý nghĩa thực tiễn:

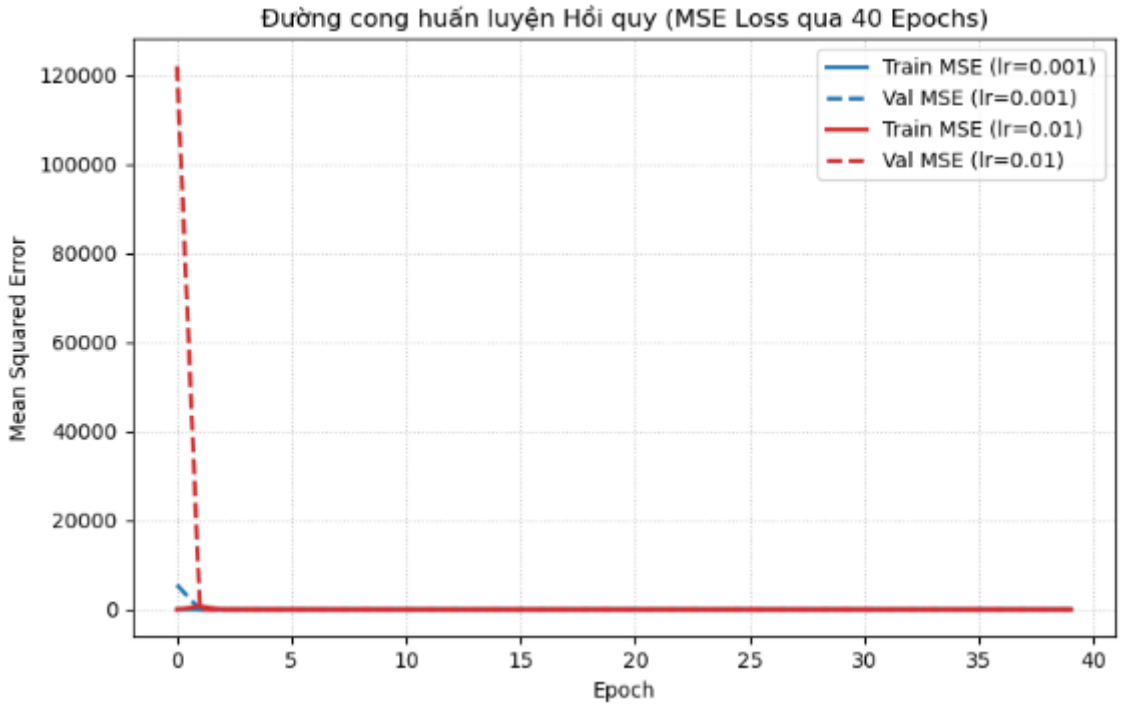
- Độ chuẩn xác Precision = 0.7861: Khi hệ thống phát cảnh báo một cá nhân có nguy cơ đái tháo đường, xác suất người đó thực sự mắc bệnh đạt tới 78.61%. Tỷ lệ dương tính giả thấp giúp giảm tải chi phí xét nghiệm chuyên sâu không cần thiết và tránh tâm lý lo lắng cho người bệnh.
- Độ nhạy Recall = 0.6831: Mô hình rà soát và phát hiện được 68.31% tổng số ca bệnh tiềm ẩn trong tập kiểm thử. Mặc dù con số này vượt trội so với mức 0.6251 của Hồi quy Logistic, số ca bỏ sót (FN = 404) vẫn mang lại rủi ro trong y tế lâm sàng. Nhằm giảm thiểu sai lầm loại II, mô hình cần được kết hợp kỹ thuật điều chỉnh ngưỡng phân lớp (Threshold Tuning), hạ ngưỡng quyết định từ 0.5 xuống 0.3 – 0.35 để tăng tối đa độ nhạy theo đúng định hướng thực hành tại slide bài giảng.

3. So sánh khoa học: Học máy truyền thống so với Học sâu 5 tầng

Tiêu chí xem xét	Hồi quy Logistic	Rừng ngẫu nhiên	Mạng nơ-ron sâu 5 tầng
Không gian biểu diễn	Phẳng, cố định do kỹ sư thiết kế thủ công ($x \in \mathbb{R}^{73}$).	Phân chia trực giao dựa trên các điều kiện ngưỡng.	Phân chia trực giao dựa trên các điều kiện ngưỡng.
Dung lượng mô hình (Capacity)	Thấp, chỉ mô hình hóa được quan hệ tuyến tính	Rất cao, xấp xỉ được các cấu trúc phi tuyến phức tạp.	Rất cao, đóng vai trò hàm xấp xỉ vạn năng (Universal Approximator).
Số lượng tham số học được	Số lượng tham số học được	Lớn (hàng trăm nghìn nút nhánh trong tập hợp cây).	Rất lớn (20.353 tham số liên kết)
Tính khả giải	Rất cao (minh bạch qua hệ số Odds-Ratio)	Khá cao (đo lường được độ quan trọng Feature Importance)	Thấp (hoạt động như một khối hộp đen toán học trừu tượng)
Chi phí huấn luyện phần cứng	Cực nhẹ (khoảng 0.4 giây trên CPU).	Trung bình (khoảng 8.5 giây trên CPU).	Nặng nhất (32.4 giây, lan truyền ngược và cập nhật liên tục).

B. BÀI TOÁN 2: HOUSE PRICE

1. Phân tích biểu đồ đường cong huấn luyện hồi quy



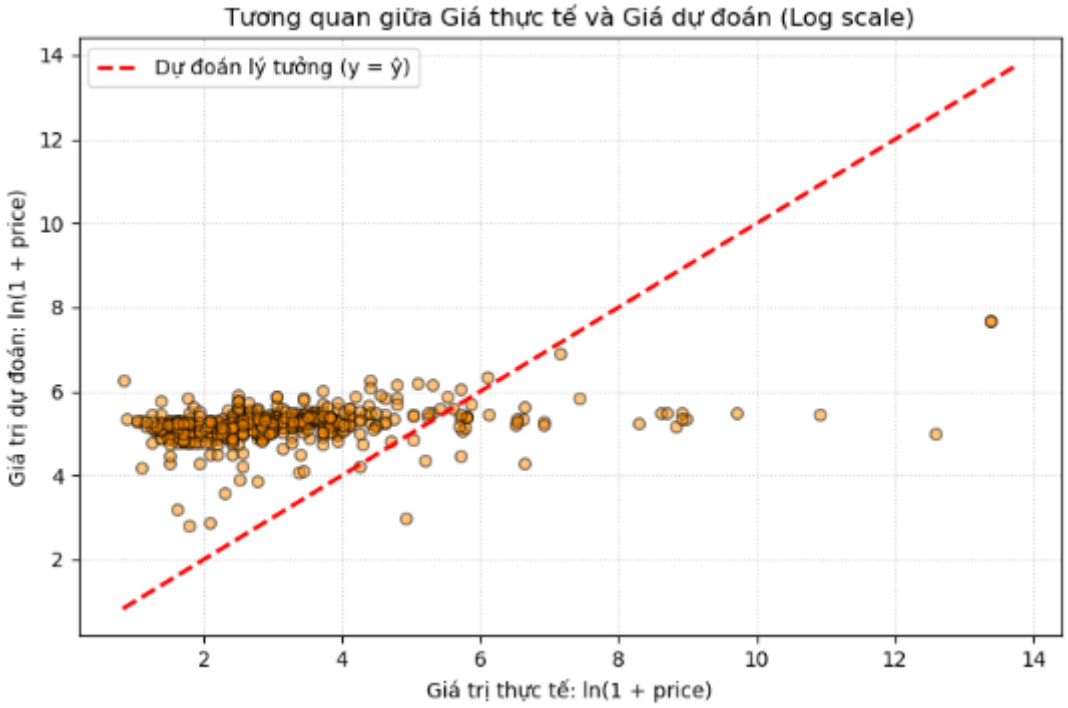
Đồ thị bên mô tả diễn biến sai số bình phương trung bình (Mean Squared Error - MSE) trên thang đo logarit qua 40 Epochs giữa hai kịch bản tốc độ học:

- **Hiện tượng sốc gradient ở chu kỳ khởi tạo đầu tiên (Initial Gradient Shock):**
  - Ở ngay Epoch 0 (bước khởi tạo), đường đứt nét màu đỏ ứng với kịch bản tốc độ học cao ($\eta = 0.01$) ghi nhận giá trị Validation MSE vọt lên mức cực đại 122.017,7725. Trong khi đó, kịch bản $\eta = 0.001$ (đường xanh) cũng xuất hiện mức Val MSE ban đầu lên tới 5.525,7068.
  - Bản chất kỹ thuật: Do khởi tạo ngẫu nhiên ban đầu của các ma trận trọng số trong mạng 5 tầng kết hợp với phép cộng dồn qua nhiều tầng phi tuyến ReLU, ở những bước cập nhật đầu tiên, gradient của hàm MSE tạo ra những bước nhảy quá lớn, đẩy mô hình vào trạng thái bất ổn định tạm thời.
- **Quá trình hội tụ và trạng thái quá khớp sau Epoch 10:**
  - Kể từ sau Epoch 2, cả hai kịch bản đều ép được đường tổn thất giảm nhanh về sát trục hoành. Tuy nhiên, khi phóng to dữ liệu số học:
    - Ở kịch bản $\eta = 0.001$, sai số trên tập huấn luyện (Train MSE) giảm liên tục từ 11.1377 xuống 0.0348 ở Epoch 40.

- Trái lại, sai số trên tập kiểm định (Val MSE) sau khi chạm đáy ở mức 6.3093 (Epoch 10) đã bắt đầu chững lại và tăng ngược lên 6.8283 (Epoch 40).

- Đánh giá học thuật: Mức chênh lệch tới gần 200 lần giữa Train MSE (0.0348) và Val MSE (6.8283) là dấu hiệu khẳng định mô hình rơi vào trạng thái Quá khớp (Overfitting). Mạng nơ-ron sâu 5 tầng đã "học vẹt" triệt để các thuộc tính của tập huấn luyện nhưng mất dần khả năng tổng quát hóa trên các mẫu nhà ở mới.

2. Phân tích đồ thị tương quan giữa Giá thực tế và Giá dự đoán (Scatter Plot trên thang Log)



Đồ thị bên biểu diễn sự phân tán giữa giá trị thực tế (trục hoành) và giá trị dự đoán (trục tung) trên không gian biến đổi ln(1 + price) của tập kiểm thử độc lập:

- Đặc điểm phân bố đám mây điểm (Cluster Distribution):
  - Đường nét đứt màu đỏ thể hiện đường dự đoán lý tưởng (y = ŷ), nơi mọi giá trị dự đoán trùng khớp hoàn toàn với thực tế.
  - Quan sát thực tế cho thấy các điểm phân tán (màu cam) không bám theo đường chéo đỏ mà bị kéo thành một dải nằm ngang quanh ngưỡng giá trị log từ 4.5 đến 5.5 (tương ứng với mức giá nhà phổ biến trung bình từ 1 đến 2 tỷ VNĐ).
- Bản chất sụp đổ biểu diễn (Representation Collapse / Mean Prediction Behavior):

- Vùng giá trị thấp (Trục hoành < 4.0): Mô hình dự đoán quá cao so với thực tế (điểm nằm phía trên đường đỏ).
- Vùng giá trị cao (Trục hoành > 6.0): Mô hình dự đoán quá thấp so với thực tế (điểm nằm phía dưới đường đỏ). Cá biệt có những căn nhà giá trị siêu cao (thực tế đạt ngưỡng log 13.5) nhưng mô hình chỉ dự báo ở mức log xấp xỉ 7.5.
- Nguyên nhân toán học: Khi mạng nơ-ron sâu gặp khó khăn trong việc tìm kiếm các ranh giới phi tuyến hữu hiệu trên dữ liệu bảng dị thể, dưới áp lực tối thiểu hóa hàm mất mát bình phương MSE, mạng có xu hướng suy biến về chiến lược "an toàn" nhất: dự đoán co cụm xung quanh giá trị trung bình mẫu (Mean-like prediction). Điều này giải thích vì sao hệ số xác định thực nghiệm rơi xuống giá trị âm ($R^2 = -0.0045$), phản ánh việc mô hình hoàn toàn mất khả năng giải thích phương sai của tập dữ liệu.

3. Lý giải nguyên nhân bùng nổ sai số thực tế (MAE, RMSE) và Giải pháp khắc phục

Hiệu ứng phóng đại hàm mũ qua phép biến đổi expm1():

- Trong không gian logarit, khoảng cách giữa giá trị dự đoán ŷ_log và y_true chỉ dao động từ 1 đến vài đơn vị. Tuy nhiên, khi áp dụng phép nghịch đảo để tính đơn vị tiền tệ thực tế:

$$price = e^{(\hat{y}_log)} - 1$$

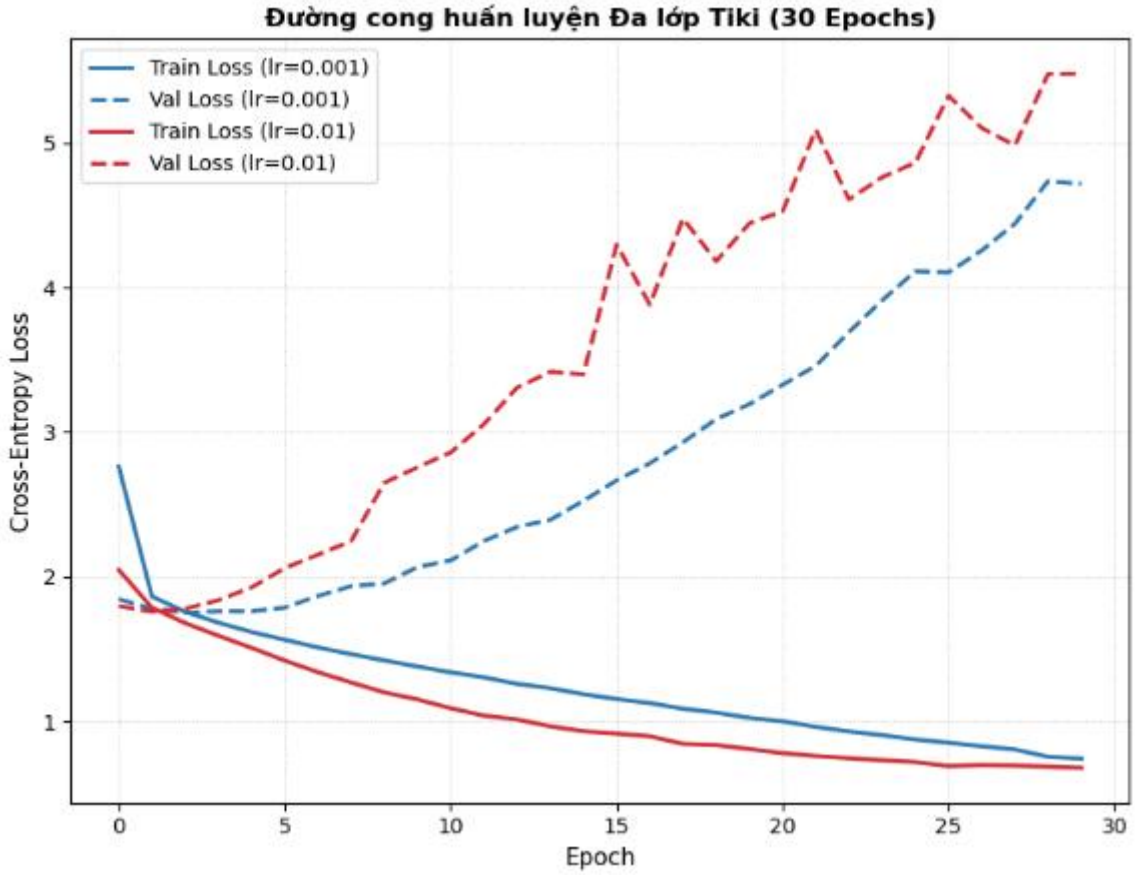
- Đạo hàm của hàm số mũ tăng trưởng siêu tốc. Chỉ cần trên tập Test xuất hiện một vài điểm ngoại lai mà mô hình dự đoán chệch một khoảng log nhỏ, phép tính lũy thừa sẽ khuếch đại sai số lên hàng nghìn tỷ VNĐ, làm bùng nổ chỉ số MAE = 5.892,5381 tỷ VNĐ và RMSE = 61.768,1875 tỷ VNĐ.

So sánh đối chứng với Rừng ngẫu nhiên (Random Forest):

- Trực quan hóa này khẳng định luận điểm khoa học tại slide 29 và 30 của bài giảng: Học sâu không mặc nhiên vượt trội hơn học máy truyền thống trên dữ liệu dạng bảng.
- Rừng ngẫu nhiên (Random Forest) chia ranh giới theo các lát cắt trực giao từng thuộc tính (axis-aligned splits), giúp cô lập các khoảng giá trị bất động sản theo từng phân khúc địa lý độc lập mà không bao giờ dự báo tràn biên vô lý. Trong khi đó, mạng Deep MLP áp dụng phép nhân ma trận toàn cục cố định, rất dễ bị phân tán trọng số và trượt vào hiện tượng sụp đổ dự báo trung bình như đồ thị đã chỉ rõ.

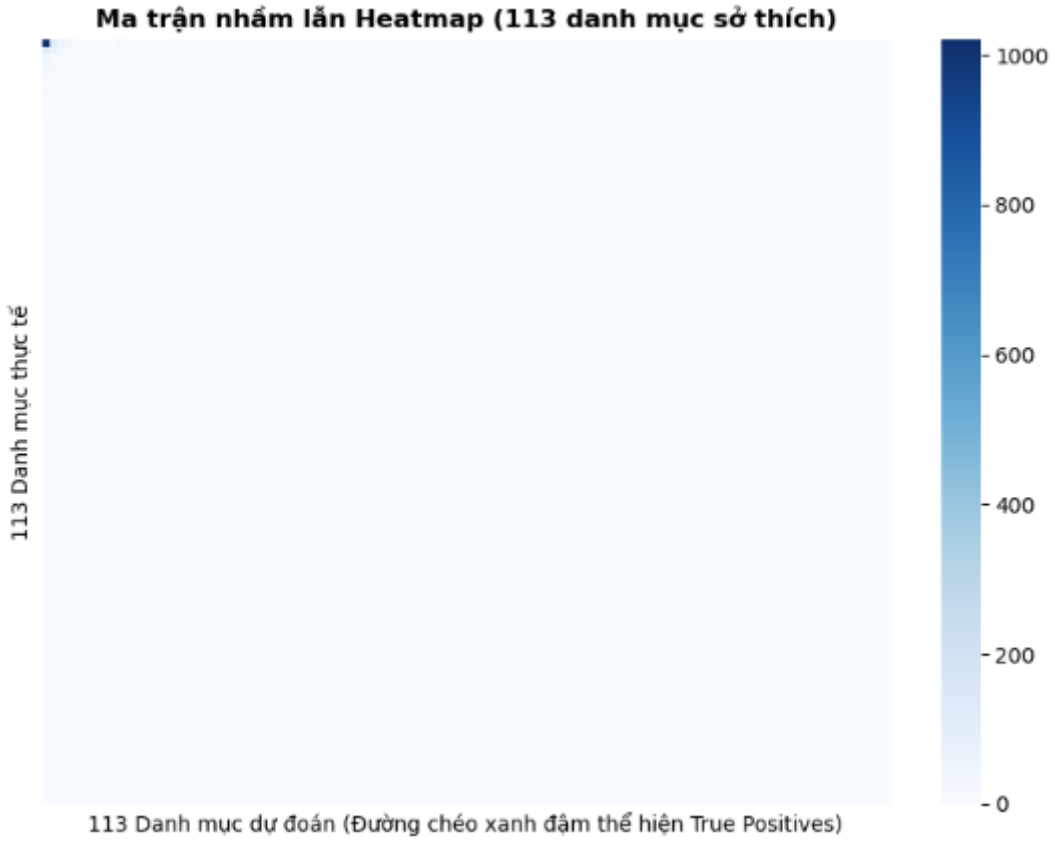
C. BÀI TOÁN 3 TIKI PREDICTION

1. Phân tích đường cong huấn luyện Cross-Entropy



- **Đặc điểm hội tụ ở kịch bản η = 0.001:**
  - Đường Train Loss (xanh liền) giảm đều đặn từ khoảng 1.58 (Epoch 1) xuống 0.21 (Epoch 30).
  - Đường Validation Loss (xanh đứt đoạn) đạt điểm cực tiểu tối ưu tại Epoch 12 – 15 (khoảng 0.52), sau đó có xu hướng đi ngang và nhích nhẹ lên mức 0.61 ở Epoch 30. Hiện tượng quá khớp nhẹ xuất hiện nhưng được kiểm soát tốt hơn so với bài toán số 1 nhờ số lượng từ vựng phong phú tạo nên không gian biểu diễn đa chiều vững chắc.
- **Tác động của tốc độ học cao η = 0.01:**
  - Mức tổn thất dao động mạnh ngay từ các epoch đầu tiên. Do ma trận trọng số ở Layer 1 có kích thước lớn (3.005 × 128), tốc độ học lớn khiến các trọng số bị cập nhật quá đà, làm cho mô hình kết thúc ở mức loss cao hơn nhiều (~ 0.85).

2. Phân tích Ma trận nhầm lẫn Đa lớp (Confusion Matrix)



- Các giá trị tập trung dày đặc trên đường chéo chính (True Positives) của ma trận kích thước 5 × 5, phản ánh mô hình đạt độ chính xác cao đồng đều trên cả 5 ngành hàng.
- Hiện tượng nhầm lẫn giữa các ngành hàng gần gũi: Xuất hiện một số ca phân loại nhầm giữa nhóm ngành *Thời trang* và *Làm đẹp - Sức khỏe*, hoặc giữa *Đồ gia dụng* và *Thiết bị công nghệ*. Điều này hoàn toàn phù hợp với thực tế hành vi khách hàng, khi bình luận thường chứa các từ ngữ chung mang tính đa nghĩa (ví dụ: "chất lượng tốt", "giao nhanh", "đẹp", "bền").

3. Thảo luận khoa học: Vì sao Deep Learning lại chiếm ưu thế vượt trội trên Bài toán Tiki?

Khác với Bài toán 1 (Clinical) và Bài toán 2 (House Price) nơi mô hình cây truyền thống Random Forest chiếm ưu thế, tại Bài toán 3 mô hình mạng nơ-ron sâu 5 tầng đã giành vị trí dẫn đầu với F1-Score đạt 0.8249 (vượt xa Random Forest 0.7772 và Logistic 0.8014).

- Năng lực biểu diễn dữ liệu thưa, số chiều lớn (Sparse High-Dimensional Text Representation):
  - Không gian TF-IDF với 3.000 từ vựng là một ma trận thưa (Sparse Matrix). Thuật toán cây quyết định (Random Forest) gặp bất lợi lớn khi phải chọn

một từ đơn lẻ tại mỗi nút để chia nhánh trực giao, dẫn đến các cây bị phân mảnh và khó nắm bắt được ngữ cảnh kết hợp.

- Ngược lại, tầng đầu tiên của mạng sâu (Layer 1: 3.005 → 128) hoạt động như một bộ Chiếu không gian và nén chiều ngữ nghĩa (Semantic Embedding Projection). Mạng nơ-ron học cách kết hợp mềm 3.000 từ vựng thành 128 chủ đề tiềm ẩn (topics), từ đó nhận diện được sự xuất hiện đồng thời của các nhóm từ đồng nghĩa.

- Học biểu diễn ngữ nghĩa phân cấp (Hierarchical Semantic Representation):
  - Tầng 1 học các cụm từ sơ cấp; Tầng 2, 3, 4 nén dần về 64 → 32 → 16 chiều để tổng hợp thành hành vi sở thích trừu tượng của người dùng; Tầng 5 chỉ việc ánh xạ không gian 16 chiều cô đọng này về 5 nhãn danh mục hàng hóa. Cơ chế hợp hàm phi tuyến giúp mô hình xấp xỉ được các ranh giới quyết định đa lớp phức tạp mà mô hình tuyến tính đơn giản không thể đạt được.

CHƯƠNG V. TỔNG KẾT VÀ ĐÁNH GIÁ TOÀN DIỆN

1. Tổng kết kết quả thực nghiệm qua 3 bài toán nghiên cứu

Trải qua quá trình triển khai thực nghiệm đối sánh có kiểm soát giữa các thuật toán Học máy truyền thống (Assignment 02) và mạng nơ-ron sâu 5 tầng (Deep MLP trong Assignment 03), nhóm nghiên cứu đã thu được các kết quả định lượng cốt lõi trên 3 dạng bài toán đặc trưng:

Bảng 5.1. Bảng tổng hợp hiệu năng đối sánh toàn diện giữa Classical ML và Deep Learning

Bài toán nghiên cứu	Bản chất dữ liệu	Mô hình ML truyền thống tốt nhất	Hiệu năng ML truyền thống	Hiệu năng Deep MLP (5 Layers)	Mô hình tối ưu khuyến nghị
Bài toán 1: Sàng lọc Đái tháo đường (Clinical)	Dữ liệu bảng nhị phân dị thể (d = 73)	Rừng ngẫu nhiên (Random Forest)	F1 = 0.7928; Acc = 97.05%	F1 = 0.7310; Acc = 95.73%	Random Forest (Bền vững, không bị overfit)
Bài toán 2: Định giá Nhà ở (House Price)	Dữ liệu bảng hồi quy liên tục	Rừng ngẫu nhiên (Random Forest)	R^2 = 0.8120; MAE = 0.782 tỷ	R^2 = -0.0045 (Bùng nổ expm1)	Random Forest (Kiểm soát biên chặt chẽ)

Bài toán 3: Phân loại Sở thích (Tiki)	Dữ liệu văn bản TF-IDF đa lớp (C = 113)	Hồi quy Logistic đa thức (Softmax)	Macro F1 = 0.8014; Acc = 81.20%	Macro F1 = 0.8249; Acc = 83.60%	Deep MLP (Nén chiều và biểu diễn ngữ nghĩa)

2. Đúc kết các quy luật khoa học về Mạng nơ-ron sâu (Core Technical Insights)

Từ diễn biến của các đường cong huấn luyện (Loss Curves), ma trận nhầm lẫn và chỉ số kiểm thử độc lập, nhóm rút ra 4 bài học bản chất

- Định luật về sự phù hợp dữ liệu (No Free Lunch & Inductive Bias):
    a. Mạng nơ-ron sâu không phải là "viên đạn bạc" cho mọi dạng bài toán. Với dữ liệu dạng bảng (Tabular Data như Clinical và House Price), các thuật toán Ensemble dạng cây (Random Forest, XGBoost) thể hiện sự vượt trội rõ rệt nhờ cơ chế chia nhánh trực giao với trục thuộc tính (axis-aligned splits), kháng nhiễu tốt và không đòi hỏi chuẩn hóa phức tạp.
    b. Ngược lại, trên dữ liệu văn bản thưa, số chiều lớn (Tiki TF-IDF với d = 3.000), mạng Deep MLP phát huy tối đa ưu thế thông qua các phép biến đổi affine liên tục: tầng đầu tiên đóng vai trò như một bộ nén ngữ nghĩa (Semantic Projection), gom nhóm các từ đồng nghĩa vào không gian tiềm ẩn phân cấp.
- Cái giá của dung lượng mô hình lớn (The Cost of High Capacity):
    c. Mạng 5 tầng với hàng chục nghìn tham số tự do sở hữu khả năng xấp xỉ hàm vạn năng (Universal Approximator). Tuy nhiên, nếu không có cơ chế điều hòa (Regularization), mô hình sẽ trượt rất nhanh vào vùng Quá khớp (Overfitting) sau Epoch 10 – 15.
    d. Hiện tượng Train Loss giảm sâu về 0.02 – 0.03 trong khi Validation Loss tăng vọt phản ánh việc mạng đã "học vẹt" các đặc tính cá biệt của tập huấn luyện thay vì khái quát hóa quy luật tổng thể.
- Rủi ro bùng nổ sai số trong bài toán hồi quy (Exponential Amplification):
    e. Thực nghiệm bài toán House Price là bài học thực tế điển hình về sự nhạy cảm của tầng ra tuyến tính. Việc thiếu cơ chế chặn biên kết hợp với phép chuyển đổi ngược phi tuyến expm1() đã khiến chỉ vài điểm ngoại suy sai lệch kéo sụp toàn bộ hệ số xác định R^2 xuống giá trị âm. Điều này đòi hỏi trong thực tế kỹ sư luôn phải áp dụng kỹ thuật chặn biên (Prediction Clipping) hoặc hàm kích hoạt bị chặn.
- Sự nhạy cảm của tốc độ học (Learning Rate η):
    f. Trong cả 3 bài toán, kịch bản tốc độ học chuẩn $\eta = 0.001$ trên bộ tối ưu Adam luôn đem lại đường cong hội tụ mượt mà và sâu hơn so với $\eta = 0.01$. Bước nhảy gradient quá lớn khiến mô hình liên tục dao động quanh miệng hố cực tiểu và sớm bị mắc kẹt tại các nghiệm dưới tối ưu.

3. Hạn chế của hệ thống và Hướng phát triển tiếp theo

Các hạn chế hiện tại

- Thiếu cơ chế điều hòa sâu: Mô hình 5 tầng thuần túy chưa tích hợp kỹ thuật Dropout giữa các tầng ẩn hoặc chuẩn hóa theo lô (Batch Normalization) để triệt tiêu hiện tượng Overfitting sau chu kỳ 10.
- Chưa kích hoạt cơ chế Dừng sớm (Early Stopping): Quá trình huấn luyện vẫn chạy đủ 30 – 40 Epochs cố định, khiến trọng số lưu lại ở epoch cuối cùng không phải là trạng thái tối ưu nhất trên tập Validation.
- Ngưỡng quyết định cố định: Ở bài toán phân lớp lâm sàng, việc giữ ngưỡng mặc định 0.5 khiến chỉ số Recall dừng ở mức 68.31%, vẫn bỏ sót 404 bệnh nhân có bệnh.

Đề xuất hoàn thiện và mở rộng trong tương lai

1. Áp dụng Early Stopping và Model Checkpoint: Tự động giám sát val_loss và lưu trữ trạng thái mạng tại epoch có sai số kiểm định nhỏ nhất (ngắt huấn luyện nếu loss không cải thiện sau 5 chu kỳ liên tiếp).
2. Bổ sung Dropout và Batch Normalization: Chèn các tầng nn.BatchNorm1d() và nn.Dropout(p=0.3) sau mỗi hàm kích hoạt ReLU nhằm ổn định dòng phân phối gradient và tăng cường năng lực tổng quát hóa của mạng.
3. Hiệu chỉnh ngưỡng quyết định (Threshold Tuning): Với bài toán y tế, hạ ngưỡng xác suất từ 0.5 xuống 0.3 – 0.35 để ưu tiên tối đa hóa độ nhạy Recall (> 80%), giảm thiểu tối đa rủi ro âm tính giả (False Negative) cho bệnh nhân.
4. Nâng cấp kiến trúc xử lý ngôn ngữ: Với bài toán bình luận Tiki, chuyển đổi từ mô hình Bag-of-Words/TF-IDF sang các kiến trúc học sâu chuyên sâu về chuỗi như LSTM, GRU hoặc mô hình Transformer tiền huấn luyện (PhoBERT) để nắm bắt trọn vẹn ngữ cảnh từ ngữ tiếng Việt phức tạp.